Università di Genova

DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Metric-based analysis to identify anomalous releases in the NPM ecosystem Software Supply Chain

Michele Frattini

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
https://www.dibris.unige.it/

Computer Science MSc
Software Security and Engineering Curriculum

Metric-based analysis to identify anomalous releases in the NPM ecosystem Software Supply Chain

Michele Frattini

Advisor: Giovanni Lagorio

Examiner: Luca Caviglione

March, 2026

Abstract

Table of Contents

Chapter 1

Introduction

Chapter 2

Background

In this chapter, we provide the fundamental information regarding Software Supply Chain (SSC), the Software Supply Chain Attacks (SSCAs), the Node Package Manager (NPM) ecosystem and the tarballs.

2.1 Software Supply Chain and Software Supply Chain Attacks

Developers tend to use ready-made software components in their projects for common functionalities, such as date parsing, HTTP request handling, or User Interface (UI), rather than implementing them from scratch.

These software components are called dependencies, and these dependencies in turn can depend on other components, thus constituting a SSC [OS23].

These components or packages are distributed in Open Source Package Repositories such as NPM, Python Package Index (PyPI) and RubyGems.

These repositories are often used to carry out attacks, as they are open and decentralized systems where anyone can freely publish a package [Ohm+20]. An attacker may inject malicious code into a legitimate package update, in order to compromise in this way all projects and packages that depend on it.

This type of attack on the SSC is called SSCA and has become increasingly common in recent years [Ohm+20].

Several attack vectors are possible [Dua+20], such as Account Compromise, which refers to compromising accounts of Package Maintainers (those who develop and manage the

packages) on the registry portal (like NPM), allowing the attacker to release malicious versions.

In SSCAs the victims are developers and users [Dua+20]. Developers can be exploited directly to steal their credentials or damage their infrastructure, and indirectly as a channel to reach users through their applications or services. Users can be exploited to steal their credentials or damage their devices.

Attackers often attempt to mask their malicious code using obfuscation techniques to prevent visual detection [Ohm+20]. The most common technique involves using a different encoding, such as hexadecimal, to hide function or variable names by renaming them with names devoid of semantic meaning, such as `_0x26499F`. In JavaScript, hexadecimal numeric literals must begin with a leading zero followed by a lowercase or uppercase letter x (`0x` or `0X`) [Con]. However, the leading underscore is inserted because JavaScript identifiers (variable or function names) cannot start with a digit. Conversely, the malicious code also has hexadecimal values without the underscore prefix when they are used simply to represent numbers and constants. This approach is simple and effective, as it does not require the use of external dependencies.

2.2 NPM Ecosystem

For the JavaScript and Node.js ecosystem, NPM is one of the largest and most active Open source package repositories, with over two million packages available [Moo+21].

All the information regarding NPM reported below has been taken from the official documentation [npm26].

The public NPM registry is a database of JavaScript packages, where developers can share and download packages.

A package is a folder containing various files that can be borrowed for use in own projects.

To resolve packages by name and version, NPM communicates with the public registry (configured by default at https://registry.npmjs.org), from which allows users to retrieve any version of a package currently available in the registry.

A NPM tarball is a gzipped package, the standard format used when packages are uploaded to a registry, and represents a package at a specific version. Instead, a Git URL refers to a package in a Git repository.

The content of the tarball may differ from the content of the Git repository, even though both represent the same package.

In fact, through the optional `files` field of `package.json`, it is possible to include only

the specified files, or exclude them if the `.npmignore` file is present.

Therefore, to ensure a fair and symmetrical comparison across packages, rather than mixing tarballs with Git repository contents, it is necessary to analyze the same type of artifact.

2.3 Tarball content

A tarball can contain any type of file, however, the following types of files are typically found within it [Pin+23]:

- **Configuration files and Metadata:**

  The `package.json` file describes the package's file or directory, and a package must contain a `package.json` file in order to be published to the NPM registry.

  In `package.json`, the name and version fields are required, and together they form an identifier that is assumed to be completely unique.

  Once a package is published with a given name and version, that specific combination can never be used again, even if it is removed with command `npm unpublish`. This is the reason why it is impossible to overwrite malicious versions once they have been released.

  Other common optional fields include the description, license, repository and dependencies on other packages.

  It is important to note that specifying a reference (a link) to a GitHub repository for a package is optional. Duan et al. [Dua+20] highlight that this lack of mandatory link represents a significant security gap because it prevents registries from verifying that the distributed package actually matches its public source code, allowing attackers to inject malicious code during the publication phase without detection.

- **Documentation and general information:**

  Frequently, documentation files, such as Markdown files like `README.md` and `CHANGELOG.md`, and the `LICENSE` text file used to specify copyright, are included inside the tarball.

- **Code files and Build Artifacts:**

  The tarball generally contains JavaScript and TypeScript files, which define the package's functional logic, but it may also include build artifacts [Gos+20].

  Build artifacts are the final product of the transformation and compilation process of a package's source code [Gos+20]. They are optimized versions of the original code automatically generated, rather than written directly by developers. Through

processes such as minification, these artifacts reduce file sizes while improving application performance. An example of such artifacts are Source Maps, files with a `.map` extension that maps between minified or transformed code and its original unmodified form, allowing the original code to be reconstructed and used when debugging [Yee].

Furthermore, this also ensures compatibility, as the code transformation during the build allows the software to run across different environments and runtimes without the end user needing to worry about the original syntax.

Since the NPM ecosystem is permissive, there is no standardized way to distinguish if a package contains build artifacts or where they are located. While common conventions suggest that folders such as `dist/`, `build/` or `lib/` contain artifacts, this structure is neither guaranteed nor verifiable a priori, as it depends entirely on the developer's choices.

Chapter 3

Related Work

Several detection tools have been developed to identify malicious NPM packages and counter SSCAs.

One of these tools is `DONAPI` [Hua+24a], which uses a combination of static analysis, based on a predefined database of sensitive Application Programming Interfaces (APIs), and dynamic analysis, to extract behavioral patterns also in order to classify malicious packages.

Another tool is `SpiderScan` [Hua+24b], which is based on graph-based behavior modeling and matching. It uses an Large Language Model (LLM) for various tasks, including the identification of sensitive APIs and the analysis of shell commands. `SpiderScan` is not the only tool that uses LLMs, as many others exist [Zah+25; Wan+25; Zha+25].

These two tools are not open source and have some limitations.

For example the dynamic analysis in `DONAPI` risks hindering its practical usefulness in real-world scenarios due to its heavyweight nature and increased slowness [Hua+24b].

Instead in `SpiderScan`, the use of prompt engineering methods that leverage the capability of LLM to detect maliciousness can be highly expensive to use in real-world scenarios.

Furthermore, these tools analyze only the last published release of the packages, without considering the analysis of more releases to obtain a vision of the evolution of the package over time.

A tool that instead analyzes the evolution of the package is the one by Zoratti [Zor25]. His research is focused on the identification of specific metrics, such as the ratio of whitespace to printable character, to identify Blank Space Trojan (BST) and Hidden Unicode Trojan (HUT) attacks in GitHub repositories.

There are countless metrics based on code analysis and file characteristics. In addition to those proposed by Zoratti [Zor25], the study conducted by Weissberg et al. [Wei+25] uses others metrics related to:

- Code Smell, such as the number of goto statements and number of function pointers
- Complexity metrics, such as the number of loops and number of local variables
- Metrics on memory management operations
- Abstract Syntax Tree (AST) metrics

Chapter 4

Analysis of Recent Attacks

In this chapter, we present in chronological order (from the oldest to the most recent) the four relevant SSCAs that occurred in 2025 in the NPM ecosystem, which we selected as case studies.

We analyze each of these attacks in detail: how many packages they compromised, how they work and what their main characteristics are, and how they differ from their benign versions.

4.1 Attack A: Malicious Windows DLL

The first attack, which occurred on July 18, 2025, infected five NPM packages, one of which, `eslint-config-prettier`, is widely used with over thirty million weekly downloads [Lab25].

A maintainer of this package fell victim to a phishing email, and the attacker, once the account was compromised, published malicious versions of the following packages:

- `eslint-config-prettier`
- `eslint-plugin-prettier`
- `synckit`
- `@pkgr/core`
- `napi-postinstall`

The tarball of these malicious versions contains a new file called `install.js` and a Dynamic Link Library (DLL) called `node-gyp.dll`, weighing 1.2 MB.

During the installation or update of one of these compromised packages, the `install.js` script is executed. This script checks the victim's operating system and, if it detects a Windows environment, it invokes `rundll32.exe`. The latter loads the `node-gyp.dll` into memory, allowing the attacker to perform remote code execution, enabling for example the installation of additional malware.

4.2 Attack B: Crypto attack

On September 8, 2025, an attacker compromised eighteen popular NPM packages, including `chalk` and `debug`, respectively the first and third most popular packages (in terms of number of stars) on NPM [Tri26].

These eighteen packages collectively have over 2.6 billion downloads each week, making this attack one of the most impactful in the recent history of the NPM ecosystem [Mat25].

Fortunately, the attack was limited in terms of financial losses, as the compromised versions of the packages remained online for a short time (about two hours) before being detected by developers and removed from NPM.

The attack was made possible by compromising a package maintainer's account (qix) via a phishing email.

After obtaining the account credentials, the attacker published malicious versions of these packages, which allowed them to steal cryptocurrencies by intercepting and manipulating users' transactions.

The malware's operation can be broken into several phases [HH25]:

1. **Injection into the Browser:** The malware integrates itself directly into the browser environment, hooking critical functions such as `fetch` and `XMLHttpRequest` (the core APIs responsible for web data transfer), and wallet-specific APIs like `window.ethereum`. This allows it to intercept web traffic and wallet interactions in real time.

2. **Monitoring for sensitive data:** Once active, the code scans network responses and transaction payloads for cryptocurrency wallet addresses and transaction details. It can recognize multiple formats including Ethereum, Bitcoin, Solana, Tron, Litecoin and Bitcoin Cash.

3. **Transaction Manipulation:** When a transaction is identified, for instance when the user is about to send cryptocurrency to a specific address, the malware replaces

the legitimate destination address with one controlled by the attacker. It uses look-alike addresses via string-matching to make substitutions less obvious, increasing the likelihood of successful theft.

In addition to replacing the destination address, the malware can also modify other transaction parameters, such as approvals or allowances, before the user's signature.

For example, if the user interacts with a smart contract, such as to swap tokens, the malware can route funds to the attacker's address, even if the user interface appears correct.

To achieve this, the malicious code contains the Ethereum addresses of several popular smart contracts:

- Uniswap V2: 0x7a250d5630b4cf539739df2c5dacb4c659f2488d
- Uniswap V3: 0xe592427a0aece92de3edee1f18e0157c05861564
- Uniswap V4: 0x66a9893cC07D91D95644AEDD05D03f95e1dBA8Af
- PancakeSwap V2: 0x10ed43c718714eb63d5aa57b78b54704e256024e
- PancakeSwap V3: 0x13f4ea83d0bd40e75c8222255bc855a974568dd4
- 1inch: 0x1111111254eeb25477b68fb85ed929f73a960582
- SushiSwap: 0xd9e1ce17f2641f24ae83637ab66a2cca9c378b9f

The malicious code is in the `index.js` file, written as a single obfuscated line of code 76,438 characters long, which also increased the file size.

The obfuscation technique used here is to encode in hexadecimal function names, variables, and constants, making the `index.js` file full of hexadecimal values such as `0x1c8` or `_0x6fd57a`, compared to the `index.js` file of benign versions.

Due to this, many behaviors and characteristics of the malware are hidden, such as the hooking of critical functions and wallet-specific APIs, the replacement of the transaction's destination address and the attacker's wallets. Consequently, these cannot be detected through static analysis.

The only detectable features in plain text are the seven smart contract Ethereum addresses, which are not hidden as they are known, public, and legitimate addresses.

4.3 Attack C: Shai-Hulud 1.0

On September 15, 2025, one hundred and eighty-seven NPM packages, including `@ctrl/tinycolor` with over two million weekly downloads, were infected by the SSCA called `Shai-Hulud` 1.0 [BC25].

The goal of this attack is to steal sensitive information from developers and NPM maintainers, such as:

- Github and NPM authentication tokens
- Access keys for Cloud services such as Amazon Web Services (AWS), Google Cloud Platform (GCP) and Azure
- System data

Its operation can be divided into these phases:

1. When the victim finishes installing or updating a package compromised by this attack, the malicious script is executed which downloads, installs and runs a platform-appropriate version of `TruffleHog` [Sec].

   `TruffleHog` is a legitimate tool, used for example to prevent the leak of secrets in code. However in this case, it is used for maliciously search for secrets in the victim's local file systems.

2. After completing the search with `TruffleHog`, the malware checks if the keys related to NPM and Github are valid and working.

   (a) If the malware finds valid NPM credentials with publish privileges, it publishes a new version for every package managed by the victim maintainer, adding this malicious code.

   In this way, the malware propagates automatically, behaving like a worm. This is the reason why so many packages were infected.

   (b) If the malware finds valid Github keys, it creates a public repository on the victim's account called `Shai-Hulud` (like the worm in the Dune novels), containing a `data.json` file with all the stolen information encoded.

The method of the first infection and the patient zero remained unknown, but it could be a case of phishing [Tea25a].

The malicious code was inserted into the packages' tarballs in the new file `bundle.js`, weighs approximately 3.7 MB and contains only a single line of code 3,734,355 characters long.

4.4 Attack D: Shai-Hulud 2.0

Just two months after the `Shai-Hulud` 1.0 attack (Section 4.3), on November 24, 2025, a new more aggressive version of the attack was identified under the name `Shai-Hulud` 2.0 [Cut25].

This time, this SSCA infected one thousand and fifty-five NPM packages within hours [Cut25].

The goal remained unchanged compared to the previous version of the attack (Section 4.3), which is to steal sensitive information such as Github and NPM authentication tokens, access keys for Cloud services, and system data.

Its operation also remained unaltered, in fact as explained in Section 4.3, the attack is based on the automated execution of `TruffleHog` [Sec], searching for secrets in the victim's local file systems.

Subsequently, if it finds valid tokens for NPM and Github, it proceeds with data exfiltration through a public repository and self-propagation.

However, this attack shows some differences and introduces several improvements [Tea25b]:

1. **Bun Runtime:** Instead of operating in a Node.js environment, in this new version, the attack uses the Bun JavaScript runtime [Sum26] to execute the malicious code more rapidly.

2. **Malicious Files:** In the tarball of the compromised packages, there are two new files: a Bun configuration file `setup_bun.js` and `bun_environment.js`.

   The first file calls `bun_environment.js`, which weighs over 10 MB and contains the malicious code in a single obfuscated line of code 10,157,373 characters long.

3. **Obfuscated Code:** The malicious code use an obfuscation technique that encodes in hexadecimal function names, variables, and constants, making the `bun_environment.js` file full of hexadecimal values and hiding the characteristics of the attack.

4. **Repository Name:** The repository created to publish the stolen information has a name of 18 random characters, with the fixed description "Sha1-Hulud: The Second Coming", confirming that it is the successor of the previous `Shai-Hulud` 1.0 attack.

5. **Destructive Behavior:** Finally, if the malware fails to exfiltrate data or replicate itself, for example because it does not find valid credentials, it attempts to completely delete the user's home folder, destroying all local files.

Chapter 5

Experimental Setup

This chapter describes the datasets used for the analyses conducted in Chapter 6 and the software tools developed to create these datasets.

5.1 Datasets

To carry out the analyses conducted in Chapter 6, we created two datasets: one Goodware (Section 5.1.1) and one Malware (Section 5.1.2).

These two datasets are a collection of metrics from a set of NPM packages.

The metrics considered, discussed in the relevant section in Chapter 6, are identical for both datasets. To compute this metric from the NPM packages, we developed a software tool described in Section 5.2.1.

Instead, the set of NPM packages and their collection are different for the two datasets. However, both datasets always have 20 versions for each package. This was done to study their behavior and evolution across versions.

Consequently, these datasets consist of a collection of CSV files, one for each metric, containing the values of that specific metric for every version of every package included in the dataset. We used the tool described in Section 5.2.1 to generate both datasets, available on GitHub[1].

The two datasets are comparable to each other as they are homogeneous, since both have packages with the same number of versions (20).

[1] https://github.com/Frazzerz/Datasets-NPM-Analysis

5.1.1 Goodware dataset

This dataset is composed of packages ranked among the 1,000 most popular in the NPM registry, in terms of number of stars [Tri26].

Out of these 1,000 packages, 371 have less than 20 versions available in the NPM registry and were therefore excluded.

The dataset includes the remaining 629 packages, with their 20 most recent available versions parsable via node-semver [Pre].

To automatically obtain these versions of these packages, we used the developed tool (Section 5.2.1), specifying the first 1,000 packages names from list of Tristan [Tri26] in a JSON file as input.

5.1.2 Malware dataset

This dataset is composed of 20 NPM packages compromised by one of the four SSCA analyzed in Chapter 4.

Each of these packages has 20 versions: one malicious and 19 legitimate.

To obtain the malicious versions of these packages, we used the developed script explained in Section 5.2.2.

The list below reports the compromised packages in the Malware dataset for each attack, including the specific malicious version number.

- For the attack A (Section 4.1) there are 3 packages:
  - `eslint-config-prettier`, version 10.1.7
  - `eslint-plugin-prettier`, version 4.2.3
  - `synckit`, version 0.11.9
- For the attack B (Section 4.2) there are 9 packages:
  - `ansi-regex`, version 6.2.1
  - `ansi-styles`, version 6.2.2
  - `chalk`, version 5.6.1
  - `color-convert`, version 3.1.1
  - `color-string`, version 2.1.1
  - `debug`, version 4.4.2

  - `supports-color`, version 10.2.1
  - `strip-ansi`, version 7.1.1
  - `wrap-ansi`, version 9.0.1

- For the attack C (Section 4.3) there are 4 packages:
  - `@ctrl/deluge`, version 7.2.1
  - `@ctrl/shared-torrent`, version 6.3.2
  - `@ctrl/tinycolor`, version 4.1.2
  - `ember-browser-services`, version 5.0.3

- For the attack D (Section 4.4) there are 4 packages:
  - `@asyncapi/avro-schema-parser`, version 3.0.26
  - `@asyncapi/bundler`, version 0.6.6
  - `@asyncapi/cli`, version 4.1.3
  - `posthog-node`, version 5.11.3

Attacks C and D involved a large number of compromised packages, respectively 187 and 1,055. For this reason, it was decided to include in the dataset a limited number of the most relevant packages for both attacks, specifically those with a high number of weekly downloads (at least over 1,000).

The Malware dataset only includes packages with at least 19 legitimate versions available in the NPM registry.

However in these attacks, there are compromised packages that have fewer than 19 versions. In such case, they were ignored and not included in the dataset (nor are they present in the list above). Specifically, two packages were excluded from attack A and nine from attack B.

For the purpose of this dataset and its analysis conducted in Chapter 6, it is not necessary that its packages are among the 1,000 most popular in the NPM registry. Anyway in the list, all nine packages from attack B are among the 1,000 most popular, while for the other attacks, none of the compromised packages fall into that category.

5.2 Software Tools

In this section, we describe the tool and script developed, along with the approaches used, to create the two datasets described in Section 5.1.

5.2.1 Tool for Analyzing NPM Packages

The `Npm-packages-analyzer` tool computes the metrics of Chapter 6 from a set of NPM package versions.

Using this tool, it is possible to construct the Goodware dataset (Section 5.1.1) and the Malware dataset (Section 5.1.2).

Implemented in Python, it is available on GitHub[2], and can be invoked from the command line in the following way:

```
python3 main.py --json <input_file>.json [--workers N] [--local]
```

The tool requires as input a JSON file containing a list of NPM packages names you want to analyze.

Optionally, the `--local` flag allows local tarballs of NPM package versions to be include in the analysis.

The tool processes one package and one version at a time, from the oldest version to the most recent one. Instead, the analysis and computation of metrics for individual files within in a version are performed in parallel, using multiprocessing. The `--workers` flag allows for specifying the number of processes to be used.

The tool's workflow can be divided into four phases:

1. **Version Retrieval:**

   First, the NPMClient component retrieves the package metadata and the list of available versions by querying the official NPM registry.

   From this list, only versions parsable via node-semver [Pre] are kept, to make them chronologically sortable.

   The tool selects the 20 most recent valid versions from the list and downloads the related tarballs for the Goodware dataset (Section 5.1.1).

   Instead, the tool selects and downloads the 19 most recent valid tarballs versions for the Malware dataset (Section 5.1.2). In this case, the twentieth version is the tarball related to the malicious version, retrieved with the developed script (Section 5.2.2) and included using the `--local` flag.

2. **Metrics Computation:**

[2] https://github.com/Frazzerz/Npm-packages-analyzer

In this phase, the tool first computes the metrics on the individual files extracted from the package version's tarball.

The computation of each metric is different, and each one is described in depth in its own section in Chapter 6.

Metrics based on the textual content of the files (Longest Line Length (Section 6.3), Unique Hexadecimal values (Section 6.4) and Unique Ethereum Addresses (Section 6.5)) are computed only on plain text files, such as JavaScript and TypeScript files, ignoring binary files such as zip, png or DLL.

To identify plain text files (and compute these metrics on it), the tool first determines the type of the file using `Magika`, a deep learning-based content-type detection tool developed by Google [Goo]. Then, the tool checks whether the detected file type is on a predefined whitelist. If not, the tool simply does not compute these metrics on that file.

Furthermore, for these metrics, the tool removes comments from JavaScript and TypeScript files using a parser from the tree-sitter library [Bru].

For the metrics Unique Hexadecimal values (Section 6.4) and the Unique Ethereum Addresses (Section 6.5) are computed by the tool using regular expressions (regex).

These regex patterns are precompiled using the `re.compile` function from the Python `re` library [Fou]. Pre-compilation improves performance by parsing the pattern once and reusing the resulting object, eliminating overhead across multiple files. Furthermore, this approach allows to specified flags, such as re.IGNORECASE to make regex case-insensitive, for instance to match both `0x` and `0X` in Unique Hexadecimal values metric.

To find all matches of a pattern within a file's content, the tool uses the `finditer` method on the compiled pattern, which returns an iterator over all matches, and is more memory efficient than collecting all results at once.

Subsequently, these metrics computed on individual files are aggregated to form metrics at the package version level. For example, to compute the package size metric (Section 6.2), the tool sums the sizes of all files contained in the tarball.

Aggregation strategies vary based on the type of metric. For instance, counts and weights are summed, the maximum value among the longest lines of the files is taken, and strings are collected into lists.

3. **Results Saving:**

   For each package, the results are saved incrementally in two CSV files.

   - `file_metrics.csv` contains one row for every file of each package version. The columns represent the metrics computed on single file.

- `aggregate_metrics_by_single_version.csv` contains one row for every package version, and the columns represent the metrics computed at the package version level.

The tool saves the results incrementally at the completion of each version's analysis, ensuring that partial results are preserved even the event of an error.

In addition to the CSV files, the tool produces a `log.txt` log file, which is also updated incrementally and serves as a copy of the terminal output.

Inside the `log.txt` file, there is general information such as the number of packages to analyze and the number of workers that will be used. Furthermore, for each package, the file also reports: the number of versions found in the registry, the status of the tarball downloads, the progress of the analysis of the selected versions and the total time for the analysis of the entire package.

4. **Dataset Generation:**

   Once the analysis is complete, the datasets explained in Section 5.1 can be created by executing the following command:

   ```
   python3 dataset.py
   ```

   The tool processes the `aggregate_metrics_by_single_version.csv` files from all analyzed packages and generates a separate CSV file for each metric. Each resulting file contains the specific metric values for every version of every analyzed package included in the analysis.

5.2.2 Download Malicious Tarballs

In this section, we describe the process and the developed script used to retrieve the tarballs of the malicious packages versions specified in Section 5.1.2.

Public Repositories of Malicious Packages

The first approach we used was to consult public datasets on GitHub, such as the one by Datadog [Dat23].

These public repositories contain malicious packages versions from different registries (e.g., NPM, PyPI, RubyGems). Their limitation is that they do not always contain specific malicious versions of compromised packages.

In fact in our case, the versions of the packages compromised by the SSCA (Chapter 4) we were looking for, were not initially present in these datasets.

Mirror registry

The alternative we used was to consult mirror registries, which are servers that replicate the content of official package repositories such as NPM, PyPI, and RubyGems [Guo+23].

Some examples of mirror registries include Huaweicloud, npmmirror, Tencent, and Alibaba.

The main characteristic of a mirror is its synchronization frequency with the official registry. This synchronization does not occur instantaneously, and the mirrors can preserve versions of packages already removed from the official registry for extended periods. The synchronization varies depending on the mirror and can be more or less frequent.

This latency makes mirrors a useful tool for retrieving malicious versions of packages after their removal from official registries.

Script to Download Malicious Tarballs

We developed a script, downloadpkgnpm.py, to automate the search and download across different mirrors.

Implemented in Python and available on GitHub[3], the script can be invoked from the command line in the following way:

```
python3 downloadpkgnpm.py <input_file>.json
```

The script requires as input a JSON file where one can specify a list of mirrors to query and a list of packages and versions to download.

For each package, the script sequentially queries the specified mirrors in search of the required version.

If the tarball is found, the script proceeds to download it. Otherwise, if the search fails on all mirrors, the operation's failure is notified.

In both cases, execution automatically continues searching for the next package version until the list is exhausted.

With this script, it is possible retrieve the malicious versions of the compromised packages used in the tool `Npm-packages-analyzer` (Section 5.2.1) to create the Malware dataset (Section 5.1.2).

[3] https://github.com/Frazzerz/Download-malicious-npm

Chapter 6

Metrics Analysis

In this chapter, we present the metrics defined for this study and the analysis of their behavior on the datasets detailed in Section 5.1. Our goal is to evaluate the effectiveness of these metrics in detecting the SSCAs discussed in Chapter 4.

6.0 Methodology

In this context, a metric is a computed measure on an NPM package version.

While the literature offers a broad range of metrics based on code analysis and file characteristics [Zor25; Wei+25], we focused on and used a set of metrics developed ad-hoc based on the most relevant characteristics of the SSCAs considered.

For this reason, we excluded metrics unrelated to the analyzed attacks, such as the number of files present in the tarball, or those related to attack characteristics hidden by obfuscation techniques, such as the hooking of wallet-specific APIs.

Each metric is discussed in a dedicated section, structured as follows:

1. **Metric Definition:** We defined the metric and explained how it was computed.
2. **Goodware Dataset Analysis:** We analyzed the distribution and evolutionary patterns of the metric within the Goodware dataset explained in Section 5.1.1. This allows us to define a baseline for the metric, that is its behavior under normal development conditions. And then, we investigated the nature of those packages that exhibit outlier values.
3. **Comparison with Malware Dataset:** Finally, we evaluated the metric's sensitivity by analyzing the values (peaks) in the compromised versions of the packages in

the Malware dataset explained in Section 5.1.2. We compare these values with the baseline of the Goodware dataset, to verify the effectiveness of the metric in detecting the SSCAs considered.

Except for the File Types metric in Section 6.1 and the Unique Ethereum Addresses metric in Section 6.5, we employ figures with the same characteristics to show the Goodware Dataset Analysis and the Comparison with Malware Dataset. To illustrate how to read these figures, we use the example figures below, which are constructed with fictional data and are not based on any real package or metric.

6.0.1 Figure Goodware Dataset Analysis

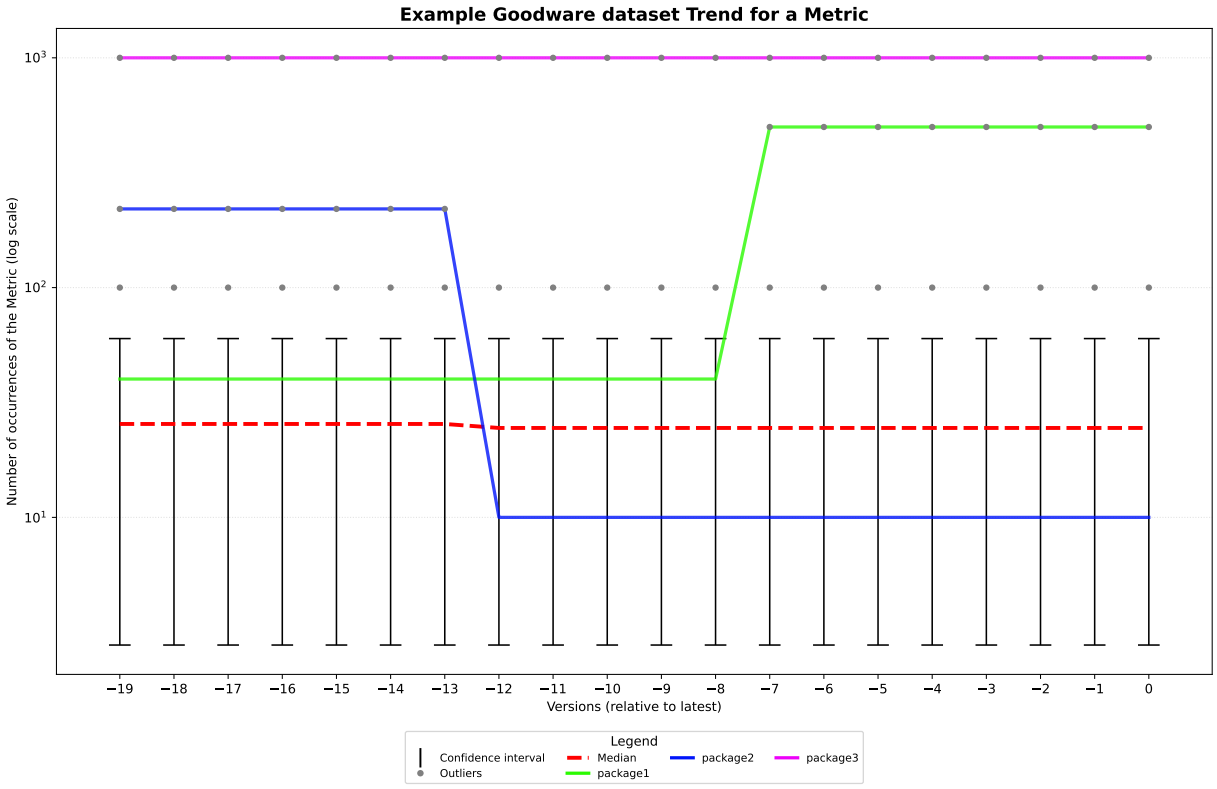

Figure 6.1: Example Goodware dataset Trend for a Metric

This type of figure, like the example Fig. 6.1, illustrates the trend across 20 versions of the metric in the Goodware dataset explained in Section 5.1.1, after excluding any packages

that are not significant in this metric, i.e. those that have a number of occurrences equal to zero in all versions.

The x-axis shows the package versions, from the least recent (-19) to the latest available (0), while the y-axis represents the number of occurrences of the metric in a logarithmic scale.

The heights of the vertical black bars represent the confidence intervals, specifically, the intervals containing 95% of the package population for a given version. Packages positioned within one of these intervals have a number of occurrences ranging between the minimum value and the 95th percentile of the population for that version.

Instead, the gray dots represent the outlier values, which are the remaining 5% of the population positioned outside the confidence interval.

A package is classified as an outlier in a version when its values exceeds the 95th percentile of that confidence interval. We heuristically chose to use the 95th percentile to define the outliers, following the approach of Patel et al. [Pat+25].

We have highlighted with colored lines some of the packages that have outlier values or values that are the upper limit of a confidence interval for one or more versions. An example of these highlighted packages are the `package1`, `package2`, and `package3` in example Fig. 6.1.

These packages, and their corresponding outlier values, exhibit atypical behavior compared to the majority of the population. For this reason, they have been analyzed individually to understand their nature and the reasons leading to such high values.

The highlighted packages may show an increasing (`package1` in Fig. 6.1), decreasing (`package2`), or stable (`package3`) trend, and for better visualization, if there are many highlighted packages, we provide separate figures for each trend.

The red dashed line represents the median value calculated for each package version. Unlike the mean, the median is not influenced by outlier values and therefore does not distort the actual behavior of the metric.

The median and the y-axis logarithmic scale were used to highlight the heterogeneity among packages in the dataset, as there are packages with a low number of values and packages with a number of values that exceed those of the majority of the population by several orders of magnitude.

6.0.2 Figure Comparison with Malware Dataset

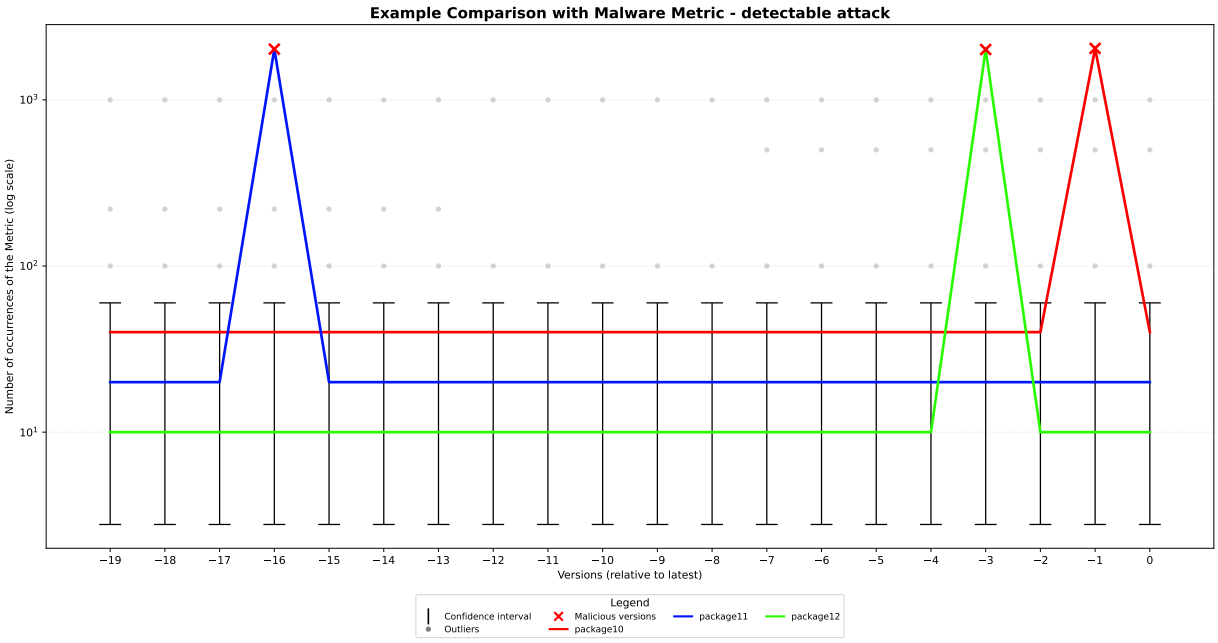

Figure 6.2: Example Comparison with Malware Metric - detectable attack

This type of figure, like the example Fig. 6.2 and the example Fig. 6.3, illustrates the trend across 20 versions of the metric in the Malware dataset (explained in Section 5.1.2) compared with the Goodware dataset (explained in Section 5.1.1).

The x-axis shows the package versions, from the least recent (-19) to the latest available (0), while the y-axis represents the number of occurrences of the metric in a logarithmic scale.

The figures show the packages of the Malware dataset for each of the four attacks with colored lines and their compromised malicious versions with red crosses.

Furthermore, the figures show the confidence intervals (vertical black bars) and the outlier values (gray dots) explained in Section 6.0.1, calculated on the Goodware dataset across the 20 versions.

If the packages of the Malware dataset overlap for a specific attack, we provide a separate figure for each package of that attack to ensure better visualization.

In these figures, we can evaluate if the metric is sensitive enough to detect the compromised versions of the packages.

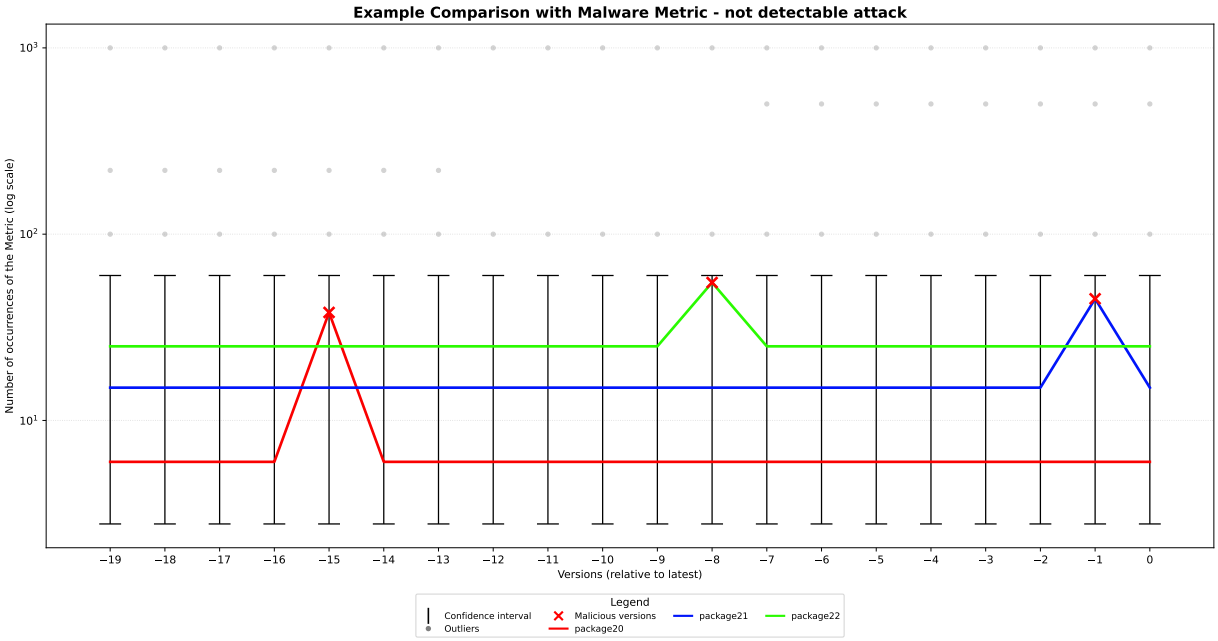

Figure 6.3: Example Comparison with Malware Metric - not detectable attack

To make malicious versions identifiable, they must be placed among the outlier values, like in Fig. 6.2.

However, if the number of occurrences in malicious versions falls within the confidence intervals, as shown in Fig. 6.3, the metric will not be indicative in detecting that attack, since the versions would be indistinguishable from the legitimate ones.

6.1 Analysis Metric 1: File Types

6.1.1 Metric Definition

The first metric we analyzed concerns the file types of NPM packages.

To compute this metric, the developed tool (Section 5.2.1) uses `Magika`, a deep learning-based content-type detection tool developed by Google [Goo].

The developed tool first determines the type of each file present in the package version and then constructs a list of all unique file types found.

This metric was taken into consideration because the attack A described in Section 4.1 introduces a malicious Windows DLL into the compromised package versions.

6.1.2 Goodware Dataset Analysis

For this metric, the Goodware dataset (explained in Section 5.1.1) contains 20 lists of file types, one for each version of every package.

We used heatmaps, Fig. 6.4 and Fig. 6.5, to visualize the distribution and evolutionary patterns of the file types of the packages in the Goodware dataset across versions.

In the heatmaps, the x-axis shows the package versions, from the least recent (-19) to the latest available (0), while the y-axis represents the file extensions.

The numbers inside the cells are the relative frequency of the extensions respect to the total number of packages in the dataset.

A value like 1.000 indicates that all packages in the dataset (100%) have at least one file of that type in that version. Conversely, a value such as 0.010 indicates that only 1% of the packages have that extension among their files.

Therefore, this number also indicates how common it is to find a given extension in the packages.

The heatmaps use dark color shades for high values and light shades for low values.

The Fig. 6.4 and Fig. 6.5 show all seventy-one file types found in the dataset, ordered from the most common to the rarest.

We used two heatmaps to better visualize the analysis:

1. **Common File Types:** Fig. 6.4 is the heatmap showing file types that have a relative frequency higher than 1% in at least one version.

2. **Rare File Types:** Fig. 6.5 is the heatmap showing all remaining types, which have a relative frequency that does not exceed 1% across the versions.

Analysis Common File Types

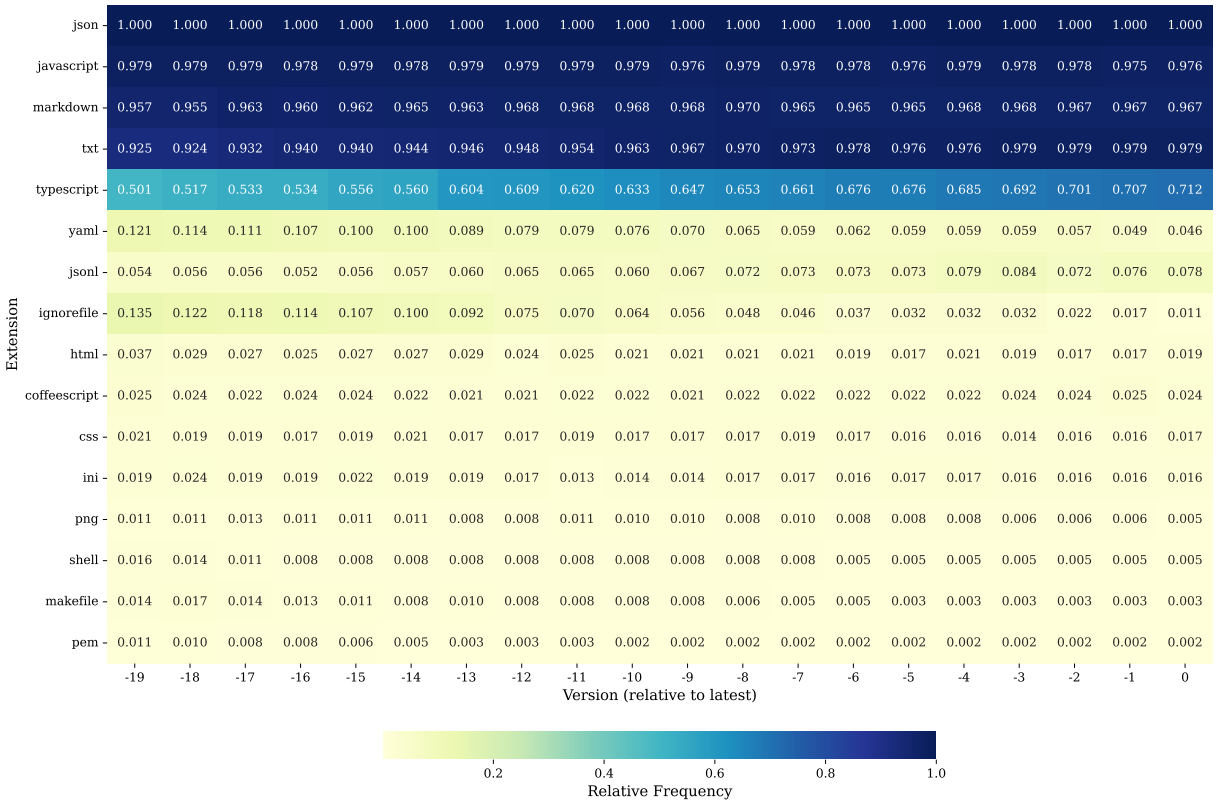

Figure 6.4: Heatmap For Common File Types

From Fig. 6.4, we can observe that:

- At least one JSON file is always present in all packages across all versions. This is because an NPM package must contain a `package.json` file in order to be published to the NPM registry [npm26].
- A JavaScript file is not always present. This can occur for several reasons:
  - **Maintainer forgetfulness**, as seen in packages such as: `@opentelemetry/api`

(version 1.0.0-rc.2), `cross-fetch` (3.2.0-alpha.0), `cheerio` (1.0.0-rc.7), `pirates` (4.0.2), `jwt-decode` (4.0.0-beta.0) and `css-what` (6.2.0).

When such human error occurs, maintainer mark the version as deprecated, release a corrected version as soon as possible, and advise users to update the package to the new version. This explains the slight decreases in relative frequency observed in some versions in the figure.

- **Project abandonment**. In some cases, the project is no longer maintained, and its latest version does not contain code, as with `@types/uuid`.
- **Purpose of the package**. Some packages do not contain executable logic, because their utility lies in the data they provide.

  For example, some packages contain only TypeScript code, such as those under the `@types` scope (e.g., `@types/node`, `@types/react`), which provide static type definitions [Def].

  Other packages, like `node-releases`, which offers a list of Node.js release versions [Tob], or `spdx-license-ids`, which provides the official list of Software Package Data Exchange (SPDX) licenses [Wat]; contain this information in JSON files without including JavaScript or TypeScript files.

  Developers use these packages as dependencies to retrieve current data without manual hard-coding.

- Other prevalent file types include markdown files, typically `README` and `CHANGELOG`, and text files, usually `LICENSE`. Small variations in their presence are due to maintainer forgetfulness.

- There is a constant growth in the presence of TypeScript files, which increase from a relative frequency of approximately 50% in older versions to about 70% in recent ones. This shift aims to provide native TypeScript support (e.g., via static type checking), ensuring better maintainability and code quality for developers. We will see this behavior for example in the `figlet` package in Section 6.2.2.

- The remaining eleven file types showed a decrease in relative frequency, never exceeding 8% in the latest versions. This is due to the practice of excluding non-essential files from the tarball to keep the package lightweight.

- Shell files are present but have decreased over time, and they are either harmless or obsolete. In the latest versions, three packages still include shell files in their tarballs: `pino`, `husky` and `jake`. Specifically, `pino` package uses a script to update its versions across its metadata files, while `husky` and `jake` use scripts to automate actions before and after a Git commit, such as check code before a commit, block it if there are errors or failing tests, or clean temporary files afterward [Typ; Eer].

Analysis Rare File Types (Outliers)

The extensions shown in Fig. 6.5 (fifty-five in total) are outliers, representing file types present in very few packages. We analyzed the most interesting extensions and their nature.

- **C and C++**. Three packages, `sharp`, `node-addon-api` and `nan`; contain C and C++ files. The `sharp` package is used for image processing (i.e., resizing, conversion, pixel operations) and relies on the C library libvips to maximize performance and speed [Ful]. To execute C code within Node.js, the use of packages such as `node-addon-api` (the more modern approach) or `nan` is required [con; VB]. This dependency is necessary because Node.js, being based on JavaScript, cannot natively execute C code. These packages act as wrappers, translating calls between the JavaScript environment and the C code.

- **gzip**. Two packages, `polished` and `moment-timezone`, included a gzip file by mistake. Specifically, the `polished` package tarball included a `.yarn` cache directory containing numerous files including `install-state.gz`, for four consecutive versions (4.0.0-beta.2 to 4.0.3). This was removed in version 4.0.4, as reported in the release notes on GitHub [HS25].

  Similarly, `moment-timezone` (version 0.5.36) erroneously included a temporary folder, `temp.bac`, which was entirely unrelated to the package's functionality. This folder contained various files types such as gzip, PE, mp3 and psd, and was subsequently removed in the following version.

- **Dockerfile**. A dockerfile appeared in a single package, `recast`, for three versions (0.21.3 to 0.21.5 included). In these versions, the `.devcontainer` folder was mistakenly included, which contains a dockerfile that configures a container with Node.js and Typescript useful only for maintainers. This folder was removed in subsequent versions.

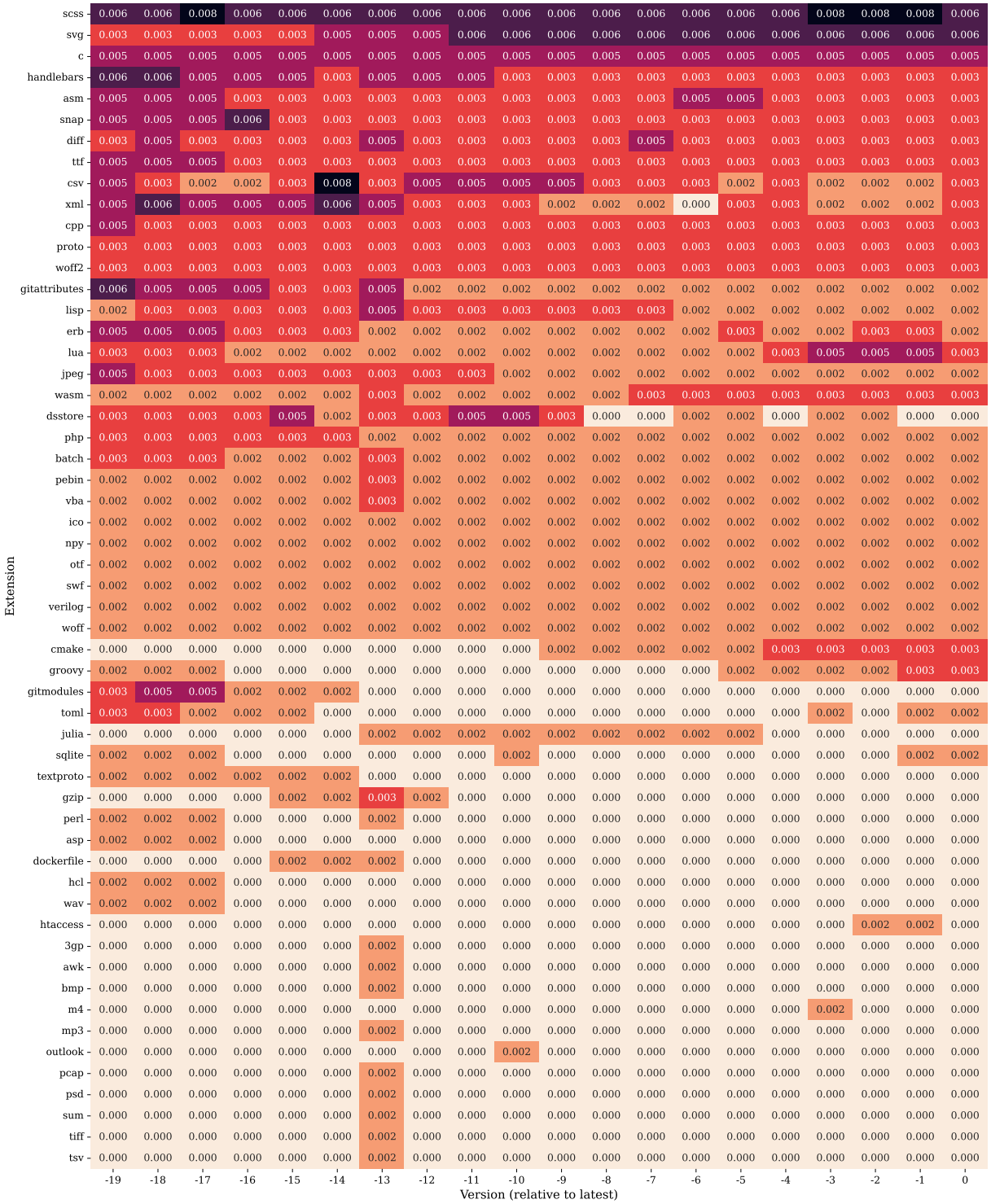

Figure 6.5: Heatmap for Rare File Types (Outliers)

6.1.3 Comparison with Malware Dataset

In attack A (Section 4.1), the attacker introduced a malicious Windows DLL into the tarball, which `Magika` correctly identifies as a Portable Executable (PE) (Pebin, PE Windows executable) file.

As shown in Fig. 6.5, the Goodware dataset contains two packages with PE files: `nodemon` (in all 20 versions) and `moment-timezone` (in one version).

- The `nodemon` package is a tool that helps develop Node.js based applications by automatically restarting the node application when file changes in the directory are detected [Sha]. Specifically, it includes `windows-kill.exe` to ensure signal event (like CTRL+C) work correctly on Windows, for instance, to allow a clean process shutdown.
- In contrast, the `moment-timezone` included a PE file by mistake within a temporary folder, as previously discussed in Section 6.1.2.

Consequently, this metric is efficient in detecting attack A because the malicious versions contain outlier file types.

However, the metric is not indicative for the other three attacks because the malicious versions modify file types that are not outliers, such as JavaScript, making them indistinguishable from legitimate versions using this metric alone.

6.2 Analysis Metric 2: Package Size

6.2.1 Metric Definition

The second metric we analyzed is the size of the NPM packages, expressed in bytes.

This metric represents the uncompressed size of the package contents. To compute this, the developed tool (Section 5.2.1) first computes the size of each extracted file, and then sums all the individual sizes to obtain the total size of the package version.

This metric was taken into consideration because all four analyzed attacks (Chapter 4) introduce or modify files that increase the package size. Specifically, attack A (Section 4.1) introduces a malicious DLL weighing 1.2 MB, attack B (Section 4.2) increases the size of existing `index.js` file by adding a malicious line of 76,438 characters, attack C (Section 4.3) adds a malicious file weighing approximately 3.7 MB, and attack D (Section 4.4) introduces a malicious file weighing over 10 MB.

6.2.2 Goodware Dataset Scenario

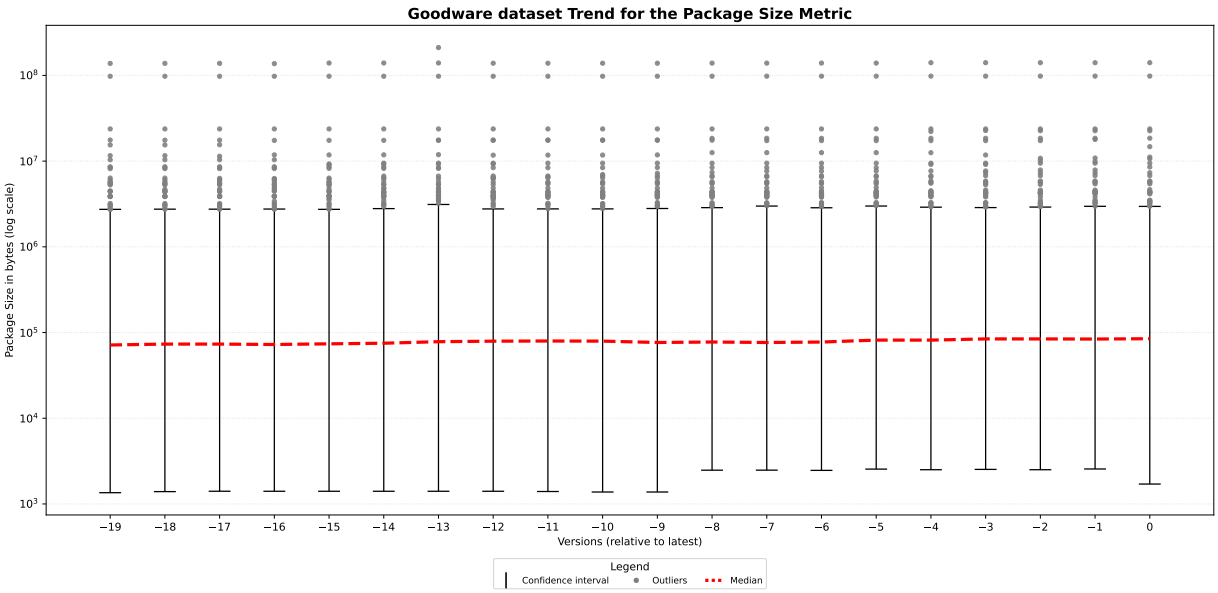

Figure 6.6: Goodware dataset Trend for the Package Size Metric

For this metric, the Goodware dataset (Section 5.1.1) contains the size for each of the 20 versions of every package.

We used four figures, Fig. 6.6 Fig. 6.7 Fig. 6.8 and Fig. 6.9, to visualize the trend of this metric across 20 versions in the Goodware dataset. The characteristics of these figures are described in Section 6.0.1.

The median grows progressively across the versions, starting from a total package size of approximately 71 KB in the older versions, up to a total size of approximately 84 KB in the most recent versions.

We studied several packages that had outlier values for one or more versions. We highlighted these nine packages with solid colored lines, categorizing them into the following figures based on their observed trends:

- **Increasing Trend:** Fig. 6.7 shows highlighted packages with an increasing trend over time.
- **Decreasing Trend:** Fig. 6.8 shows highlighted packages with a decreasing trend.
- **Stable Trend:** Fig. 6.9 shows highlighted packages with a stable trend.

Analysis Outliers Packages: Increasing Trend

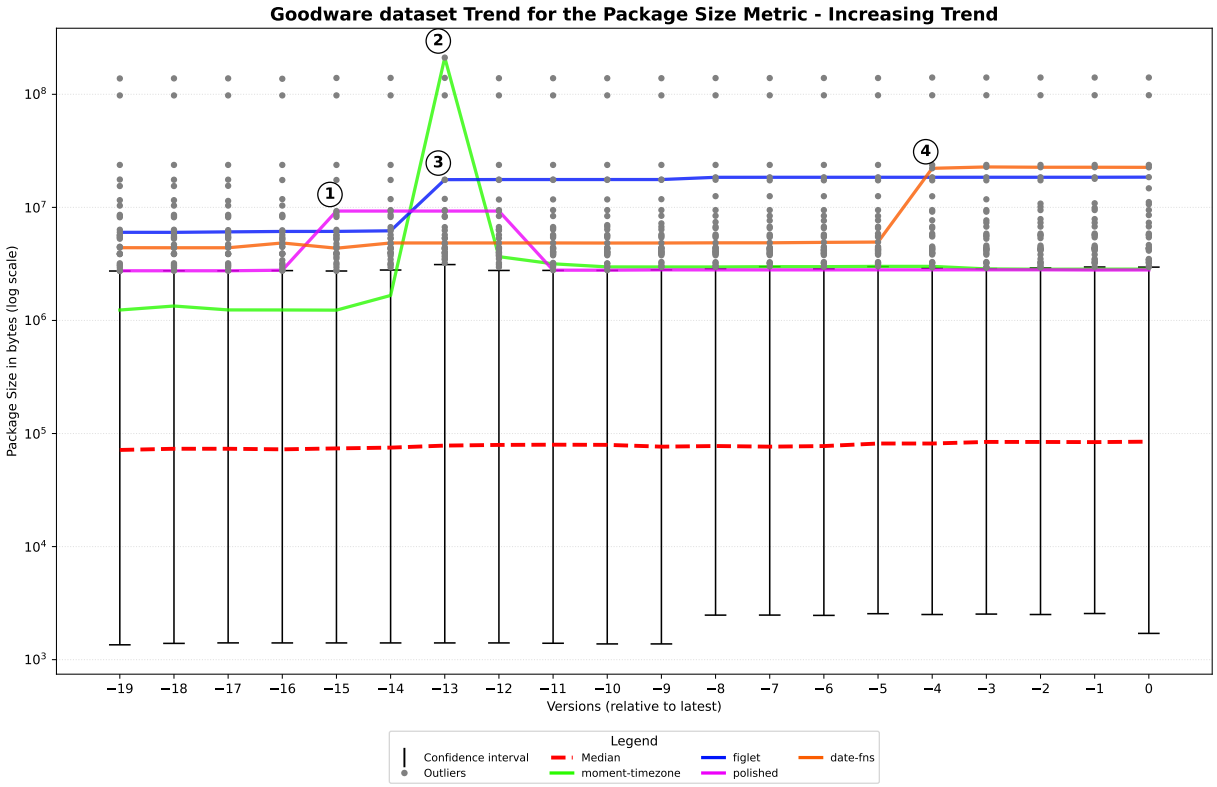

Figure 6.7: Outliers package Increasing Trend for the Package Size Metric in Goodware dataset

These four packages, `polished`, `moment-timezone`, `figlet` and `date-fns`, had outlier values and also experienced a significant increase in size from one version to the next.

- **Polished:** Represented by the solid pink line in Fig. 6.7, `polished` is used for styling styling in JavaScript [HS]. As seen at point 1 in the figure, it had a massive increase in package size, going from about 2.7 MB to 9.2 MB starting from version 4.0.0-beta.2 for four consecutive versions. In these versions, as previously seen in Section 6.1.2, the developers erroneously included a `.yarn` cache directory in the final tarball distributed to the public, which contained numerous files. In version 4.0.4, the folder was removed from the tarball, as reported in the release notes on GitHub [HS25], returning to its previous dimensions.

- **moment-timezone:** Represented by the solid green line in Fig. 6.7, the `moment-timezone` package at point 2 in the figure experienced a significant increase in size, skyrocketing from about 1.6 MB to 211 MB. As previously seen in Section 6.1.2, in version 0.5.36, the developers erroneously included a temporary folder containing various files, making this version the heaviest in the entire Goodware dataset. In the subsequent update, the folder was removed, returning to its previous size.

- **figlet:** Represented by the solid blue line in Fig. 6.7, `figlet` is used to create ASCII art text from normal text strings, transforming letters into large stylized characters composed of ASCII symbols [Gil]. As seen at point 3 in the figure, it had a significant increase in size from about 6.2 MB to 17.5 MB. In version 1.9.0, it underwent a heavy refactory to use TypeScript and modern tooling, adding approximately 300 files, as reported in the release notes on GitHub [Gil26].

- **date-fns:** Represented by the solid orange line in Fig. 6.7, `date-fns` provides a toolset for manipulating JavaScript dates in a browser and Node.js [tea]. As seen at point 4 in the figure, it had a significant increase in size from about 5 MB to 22 MB. In version 3.6.0, the developers added 400 new files for compatibility reasons with older browsers, according to GitHub release notes [tea26].

Analysis Outliers Packages: Decreasing Trend

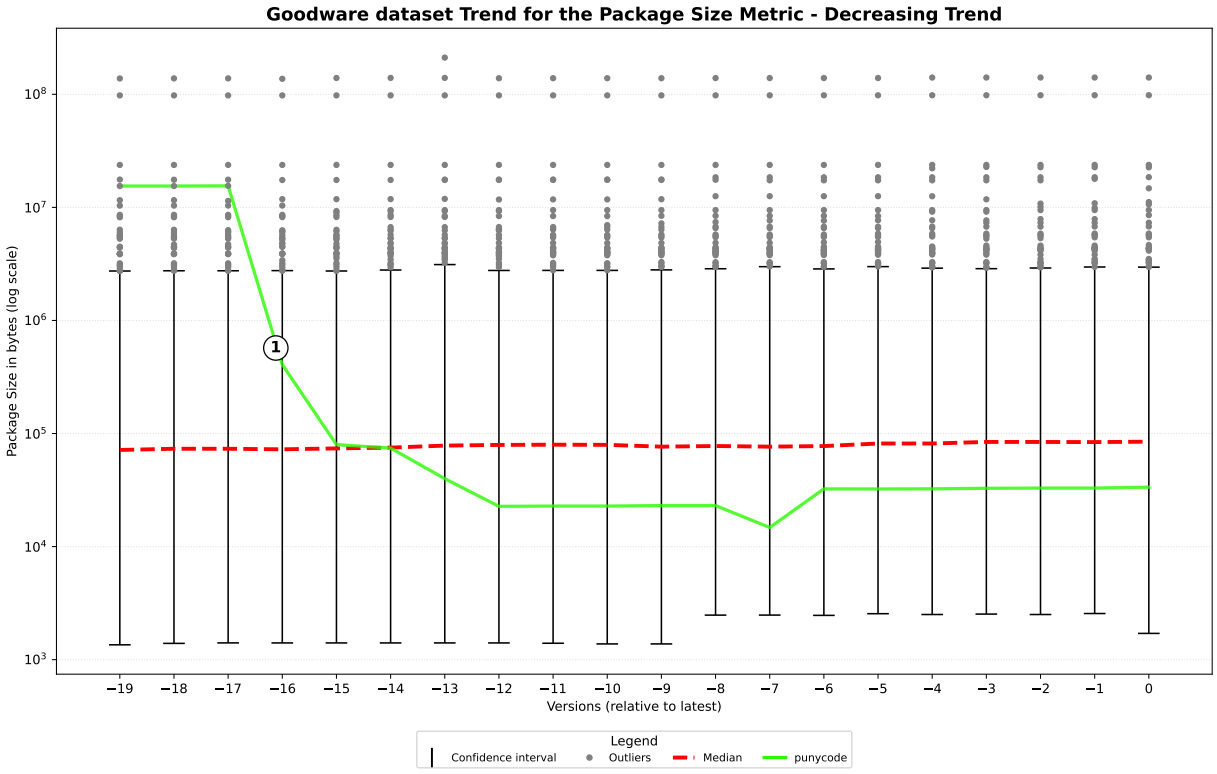

Figure 6.8: Outliers package Decreasing Trend for the Package Size Metric in Goodware dataset

Only the `punycode` package had a decrease sufficient to move from an outlier value to a value within the confidence interval from one version to the next. As can be seen in Fig. 6.8, `punycode` (the green line) in the first three versions had a total package size of 15.5 MB, and contained about 600 files. Starting at point 1 in the figure, the package has been optimized over the releases to include only the essential files, reducing its overall size, and containing only five files for a total size of about 33 KB in the latest versions.

Analysis Outliers Packages: Stable Trend

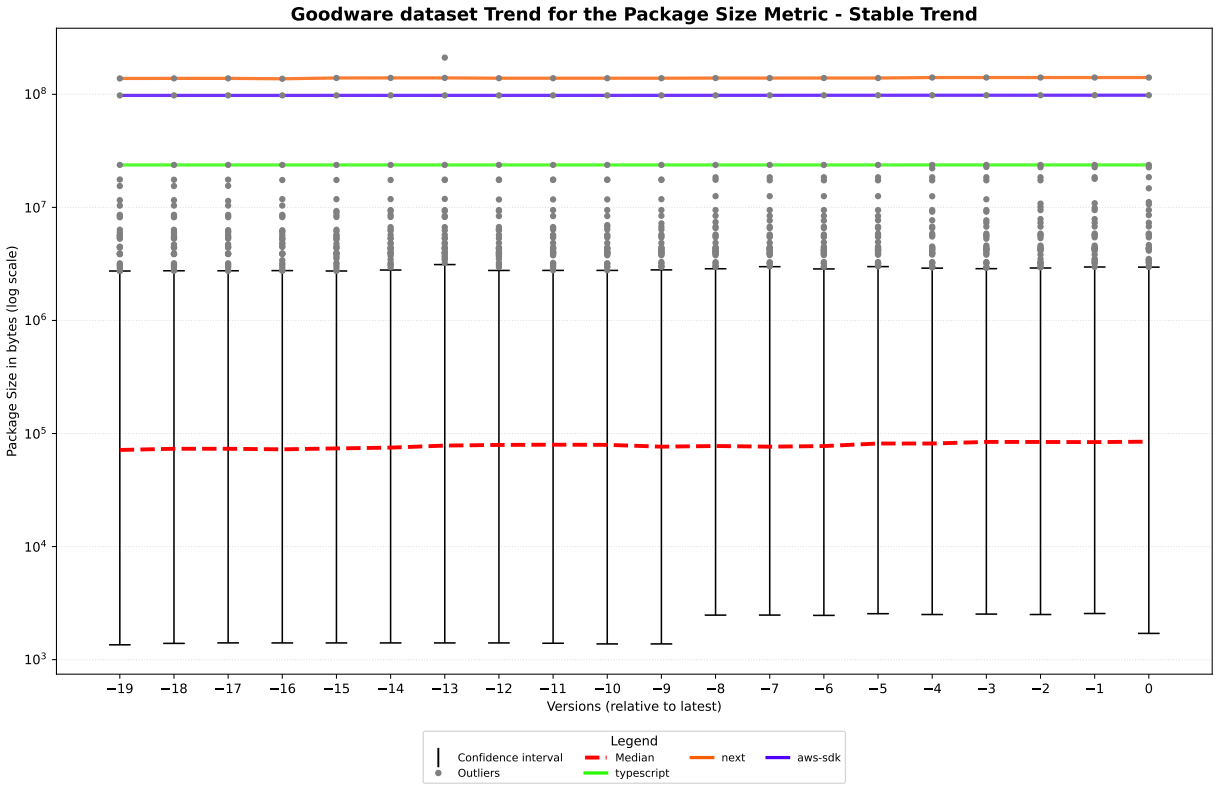

Figure 6.9: Outliers package Stable Trend for the Package Size Metric in Goodware dataset

The remaining three highlighted packages had outlier values that remained constant across all 20 versions, without significant increases or decreases. The packages exhibiting this trend are: `next`, `aws-sdk` and `typescript`.

- **next:** This package is a React full-stack framework [Ver]. Represented by the solid orange line in Fig. 6.9, the package has a size of 140 MB across all versions. It is the most extreme outlier in all versions except in version -13, where it was surpassed by `moment-timezone`, as seen in Section 6.2.2.
- **aws-sdk:** This is the official package for interacting with AWS cloud services [Ama]. Represented by the solid purple line in Fig. 6.9, the package has a size of 98 MB across all versions.

- **typescript:** This package provides the official compiler and toolset for the TypeScript language [Mic]. Represented by the solid green line in Fig. 6.9, it maintains a constant size of 23.7 MB across all versions.

6.2.3 Comparison with Malware Dataset

We analyzed when this metric is effective for detecting these four attacks (described in Chapter 4) and when it is not indicative. To do so, we used a specific figure for each attack, Fig. 6.10, Fig. 6.11, Fig. 6.12 and Fig. 6.13, as explained in Section 6.0.2.

Attack A: Malicious Windows DLL

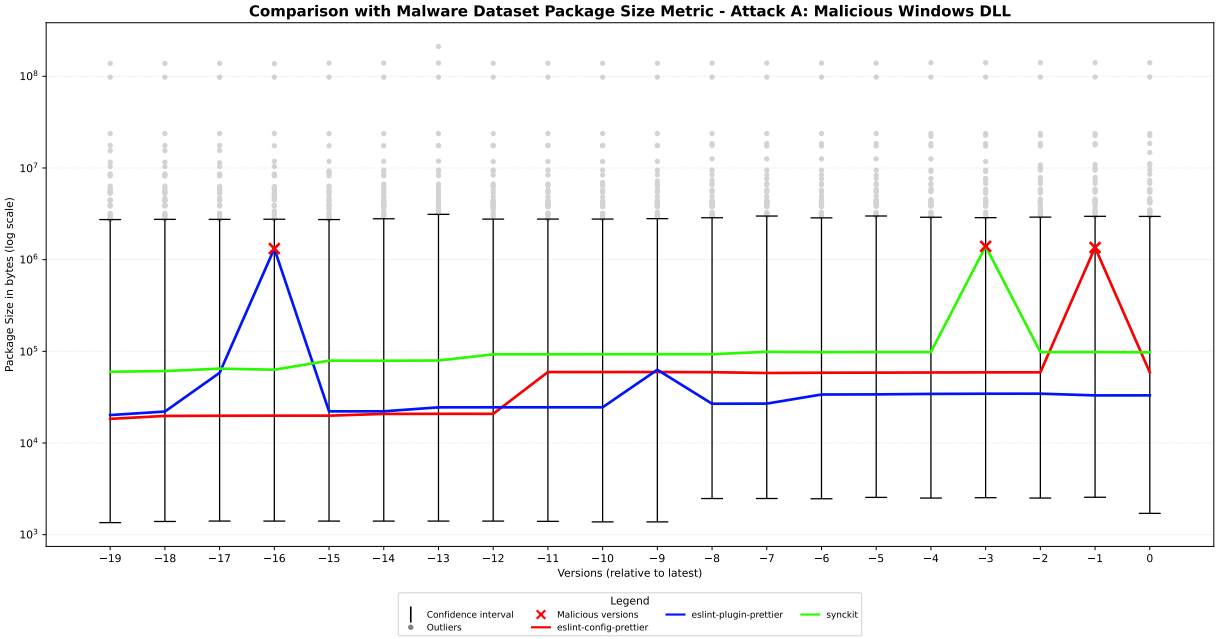

Figure 6.10: Comparison with Malware Dataset Package Size Metric for Attack A

In the attack A (Section 4.1), the attacker inserted a malicious DLL weighing 1.2 MB into the tarball. As shown in Fig. 6.10, the compromised versions of the three packages from the Malware dataset affected by this attack do not reach outlier values. Consequently, the metric is not able to detect this specific attacks and results as not indicative.

Attack B: Crypto attack

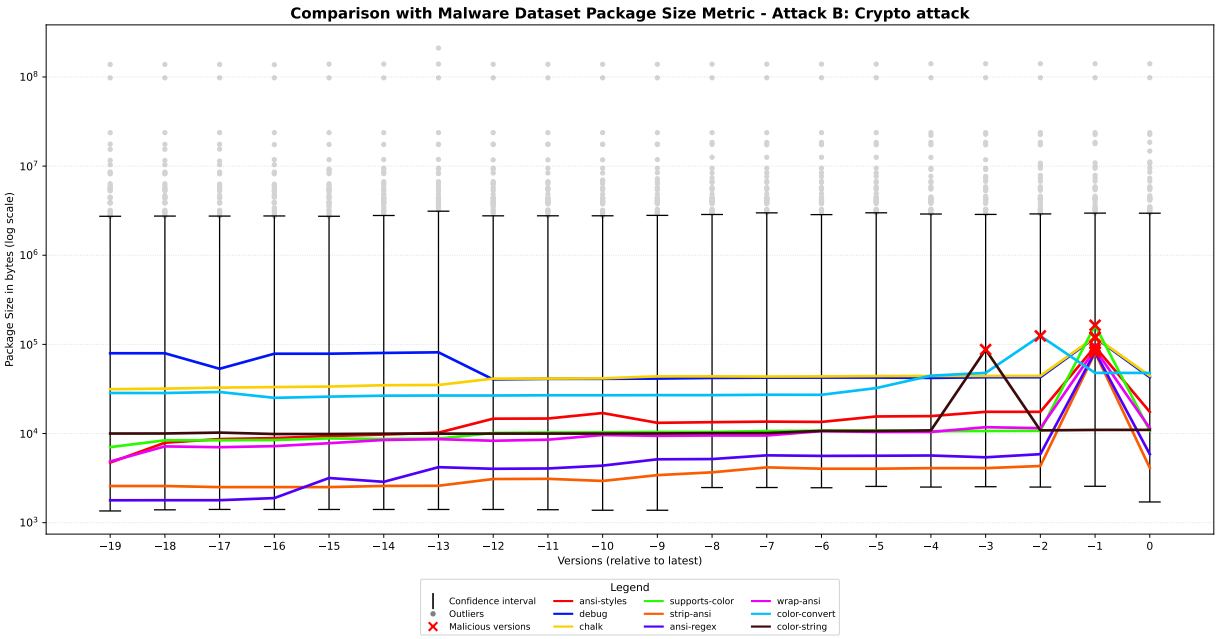

Figure 6.11: Comparison with Malware Dataset Package Size Metric for Attack B

Attack B (Section 4.2) increases the size of existing `index.js` file by adding a malicious line of 76,438 characters. Even in this case, the increase in size of the versions of the nine packages compromised by this attack is not sufficient to be among the outlier values, as can be seen in Fig. 6.11. For this reason, the metric is not indicative for detecting this specific attack, as the malicious versions are indistinguishable from legitimate ones.

Attack C: Shai-Hulud 1.0

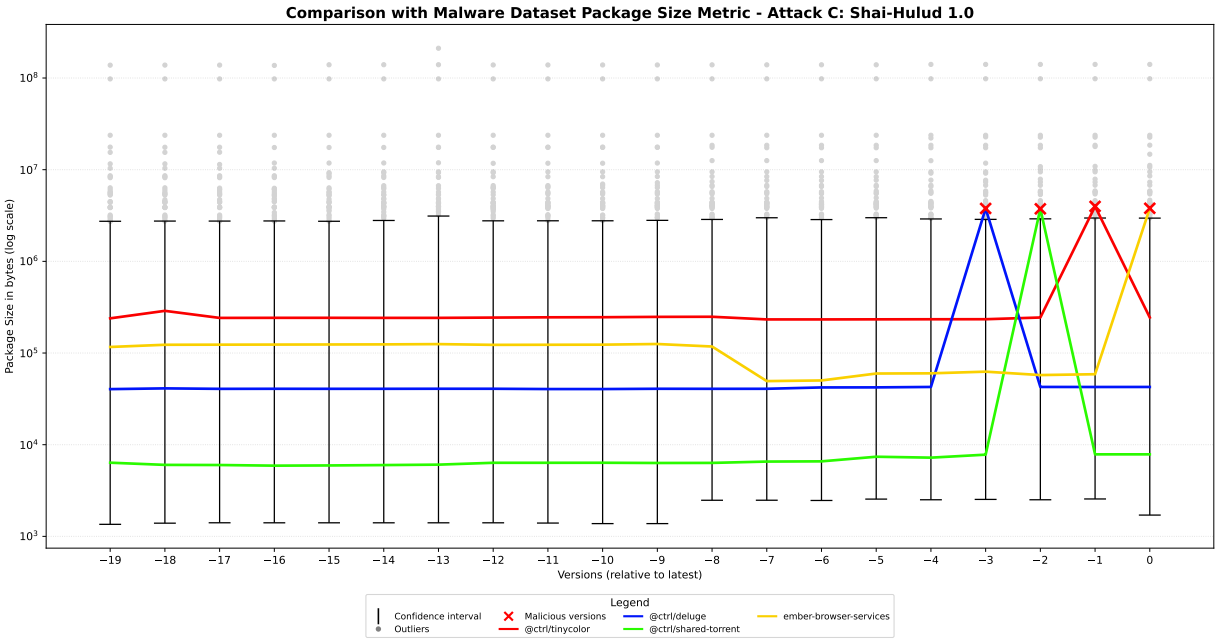

Figure 6.12: Comparison with Malware Dataset Package Size Metric for Attack C

The attack C (Section 4.3) which adds a malicious file weighing approximately 3.7 MB, succeeds in placing the compromised versions of the packages affected by this attack among the outlier values, as can be seen in Fig. 6.12. Therefore, the metric is effective for detecting this specific attack.

Attack D: Shai-Hulud 2.0

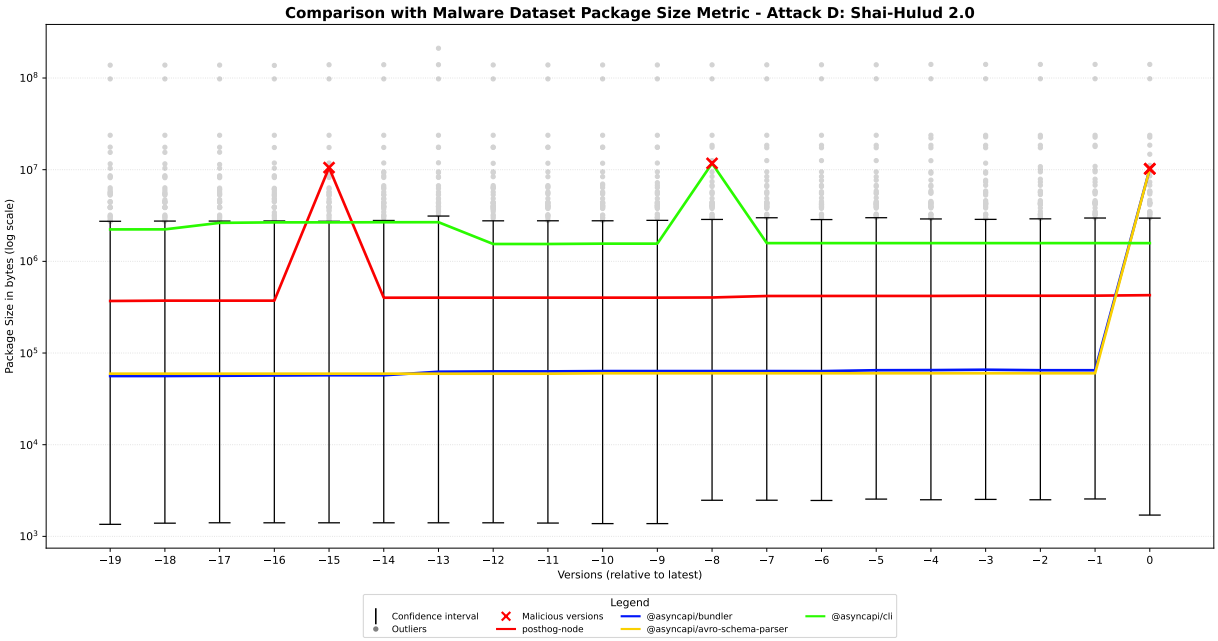

Figure 6.13: Comparison with Malware Dataset Package Size Metric for Attack D

Also for attack D (Section 4.4) that introduces a malicious file weighing over 10 MB, the metric is indicative for detecting this specific attack, as can be seen in Fig. 6.13.

Summary of Comparison with Malware Dataset

In summary, the metric is efficient in detecting the attack C (`Shai-Hulud` 1.0 Section 6.2.3) and the attack D (`Shai-Hulud` 2.0 Section 6.2.3) because, in both cases, the malicious versions have a package size that qualifies as outlier values.

Instead, the metric is not indicative for detecting the attack A (Malicious Windows DLL Section 6.2.3) and the attack B (Crypto attack Section 6.2.3) because, in both cases, the malicious versions have a package size that position them within the confidence intervals, rendering them indistinguishable from legitimate versions.

6.3 Analysis Metric 3: Longest Line Length

6.3.1 Metric Definition

The third metric we analyzed is the length of the longest line found in the plain text files of the NPM packages, expressed in number of characters.

To compute this metric, the developed tool (Section 5.2.1) measures the length of each line in each plain text file by splitting the content at newline characters. The longest line length for a single file is defined as the maximum length among all its lines. The tool then stores the longest line length for each plain text file present in the tarball, and subsequently, it take the maximum value among these to compute the longest line length metric for the package version.

This metric was taken into consideration because the attacks B (Section 4.2), C (Section 4.3) and D (Section 4.4) introduce malicious code written as a long single line. Specifically, attack B adds a single malicious line of 76,438 characters to the existing `index.js` file, attack C adds a malicious file containing a single line of 3,734,355 characters, and attack D introduces a malicious file containing a single line of 10,157,373 characters.

6.3.2 Goodware Dataset Scenario

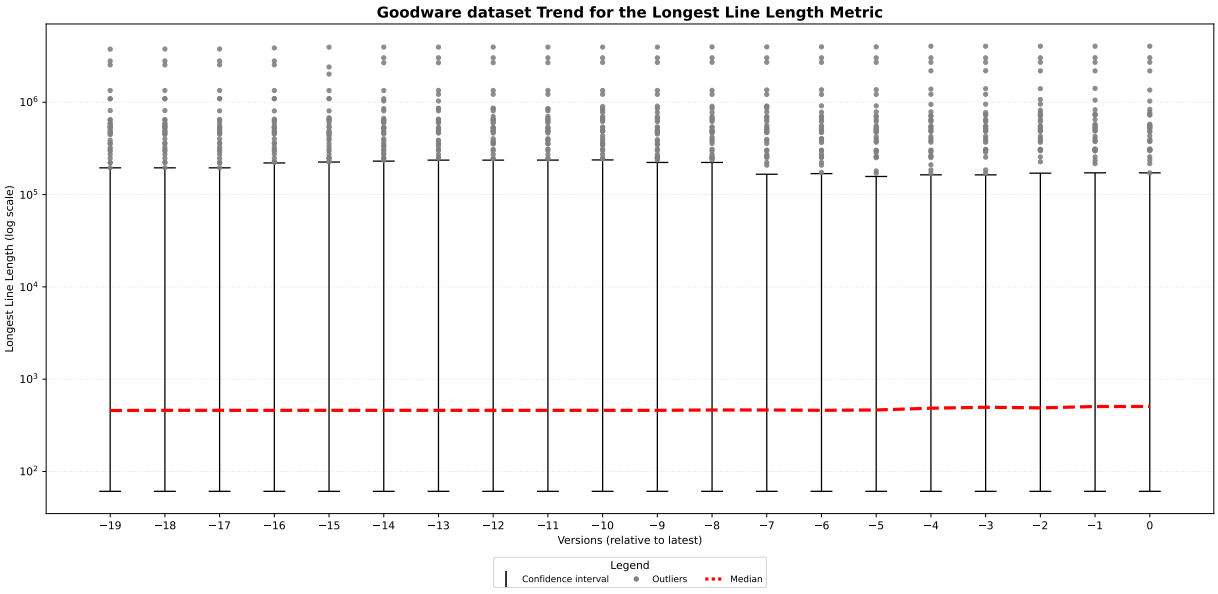

Figure 6.14: Goodware dataset Trend for the Longest Line Length Metric

For this metric, the Goodware dataset (Section 5.1.1) contains the longest line length for each of the 20 versions of every package.

The Fig. 6.14 and the Fig. 6.15, show the trend of this metric across 20 versions in the Goodware dataset. The characteristics of these figures are described in Section 6.0.1. In both figures we can observe that the median remain stable around 460 characters across all versions.

An important characteristic of this metric is that it is influenced by the presence of build artifacts contained in the tarballs. These are files automatically generated by build tools, and they can sometimes have extremely long lines.

In this analysis, we chose to consider the entire contents of the tarballs without excluding any file or directory a priori. This is because an attacker could inject malicious code into any file or directory within the tarball, and excluding specific file types or folders a priori would have limited the analysis of this metric.

Analysis Outliers Packages

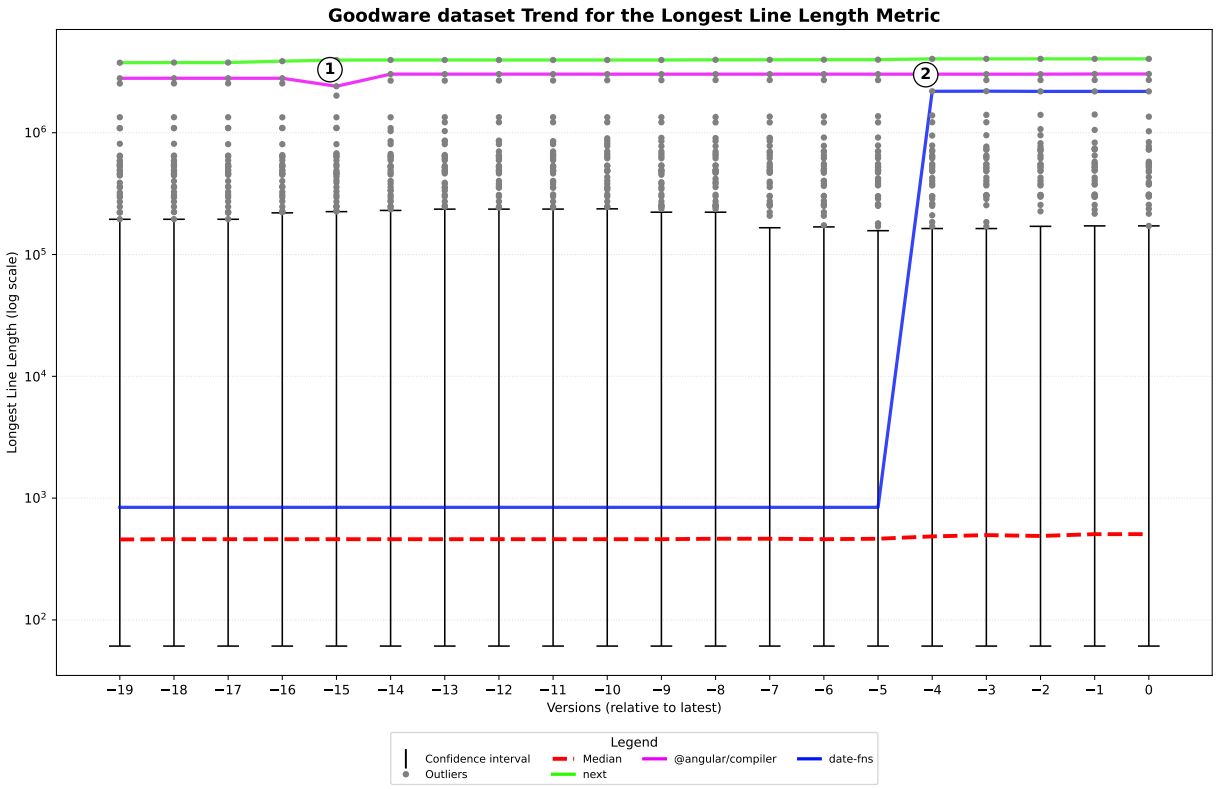

Figure 6.15: Outliers package Trend for the Longest Line Length Metric in Goodware dataset

In Fig. 6.15, we highlighted several packages that exhibit outlier values in their versions precisely due to the presence of build artifacts within their tarballs.

- **next:** The `next` package, a React full-stack framework [Ver], is represented in Fig. 6.15 by the solid green line. In all its versions, the length of its longest line among all plain text files is 4,041,545 characters, making it the most extreme outlier in the Goodware dataset across all versions.

  Specifically, the file containing this line is `app-page-turbo-experimental.runtime.dev.js.map`, a source map consisting of a single line only.

  `Next` also contains other build artifacts, such as the file `app-page.runtime.dev.js`, a JavaScript file generated by build tools, which contains 40 lines, all very long, with the longest line being 513,963 characters.

Both files are located in the `dist` folder; however, it is not guaranteed that build artifacts are always present in this folder, nor in any specific folder, as their location depends entirely on the developers' choices.

- **@angular/compiler:** In fact, in the `@angular/compiler` package represented in Fig. 6.15 by the solid pink line, in all versions the longest line is always from the `compiler.mjs.map` file contained in the `fesm2022` directory.

  This package is the official compiler of the Angular framework, which is responsible for tasks such as translating templates with Angular syntax [Tea]. It has the second most extreme outlier value in the Goodware dataset after `next`.

  It can be observed that, although the length of the longest line in this file remains constant at around 3,000,000 characters, build artifacts may vary more or less significantly from one version to the next. In fact, as indicated at point 1 in Fig. 6.15, in version 21.0.0-rc.0 the longest line length of `compiler.mjs.map` decreased to about 2,500,000 characters. These possible variations are attributable to the build process that generates such files.

  `@angular/compiler` also contains other build artifacts, such as the file `compiler.mjs`, a JavaScript file generated by build tools, containing approximately 30,000 lines, with the longest line being 14,825 characters.

- **date-fns:** Finally, we highlighted the package date-fns in Fig. 6.15 with the solid blue line, to show how build artifacts can be introduced in a new version without having been present in previous ones.

  As indicated at point 2 in the figure, and already discussed in Section 6.2.2, in version 3.6.0 the developers added new files for compatibility reasons with older browsers. Among the files added in this version is `cdn.js.map` in the `locale` directory containing 10 lines with the longest line being 2,188,784 characters. This file has the longest line among all plain text files in the package. `date-fns` also includes other build artifacts, such as the file `cdn.js` containing 10 lines with the longest line being 777,296 characters.

6.3.3 Comparison with Malware Dataset

We analyzed when this metric is effective for detecting these four attacks (described in Chapter 4) and when it is not indicative. To do so, we used a specific figure for each attack, Fig. 6.16, Fig. 6.17, Fig. 6.18 and Fig. 6.19, as explained in Section 6.0.2.

Attack A: Malicious Windows DLL

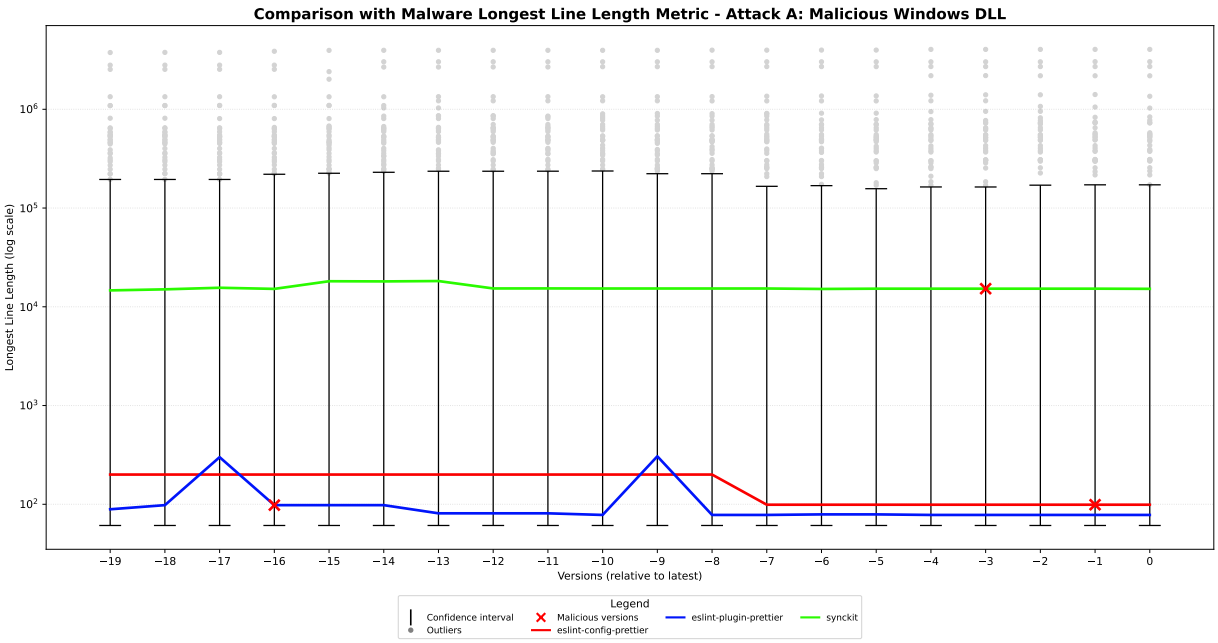

Figure 6.16: Comparison with Malware Dataset Longest Line Length Metric for Attack A

In the attack A (Section 4.1), the attacker included a plain text file, `install.js`, which however did not contain the longest line in the compromised versions. Conversely, the malicious DLL added to the tarball was correctly excluded from the longest line length computation, as it is not a plain text file. Consequently, the longest line length of the compromised versions remained unchanged compared to the legitimate versions, as shown in Fig. 6.16, making this metric ineffective for detecting this specific attack.

Attack B: Crypto attack

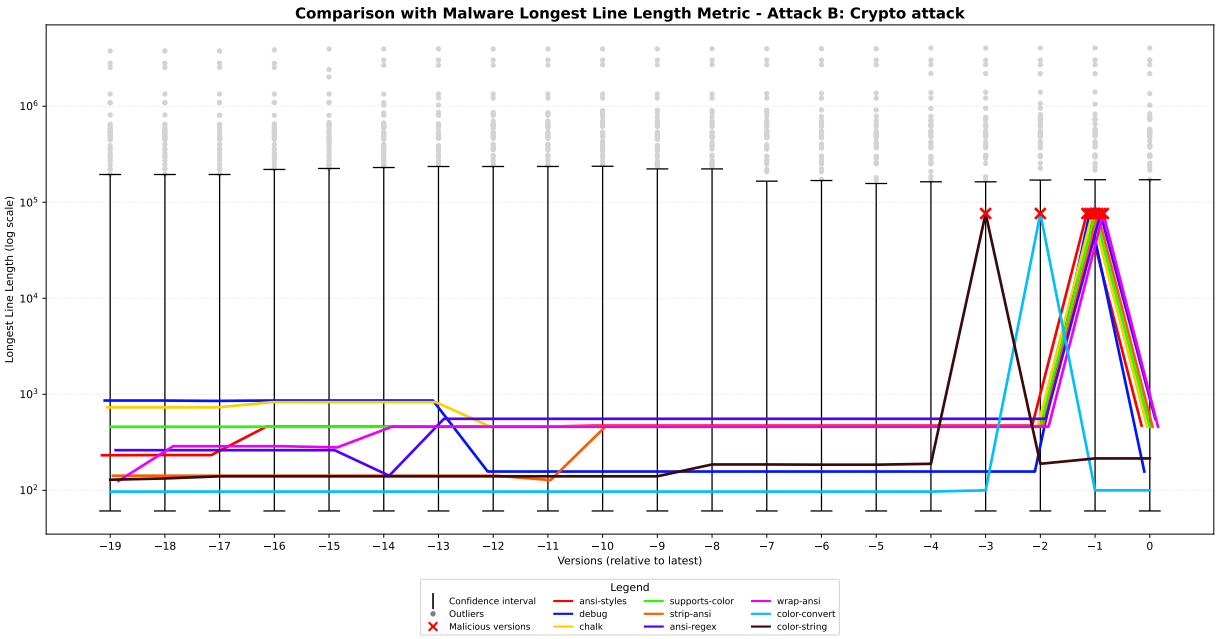

Figure 6.17: Comparison with Malware Dataset Longest Line Length Metric for Attack B

Attack B (Section 4.2) adds a single malicious line of 76,438 characters to the existing `index.js` file. As shown in Fig. 6.17, the compromised versions exhibit a significant increase in the metric, but it is not sufficient to position them among the outlier values. For this reason, the metric is not indicative for detecting this specific attack, as the malicious versions are indistinguishable from legitimate ones.

Attack C: Shai-Hulud 1.0

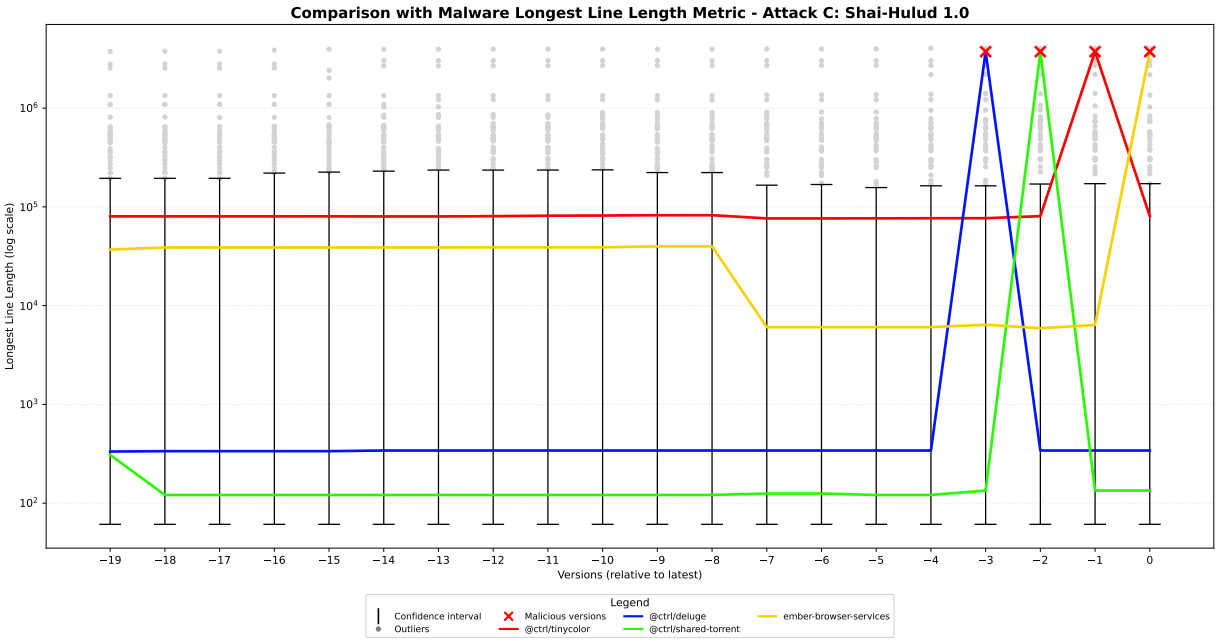

Figure 6.18: Comparison with Malware Dataset Longest Line Length Metric for Attack C

In the attack C (Section 4.3), the attacker introduced a malicious file named `bundle.js` containing a single line of 3,734,355 characters. As shown in Fig. 6.18, the malicious versions succeed in placing themselves among the outlier values, therefore the metric is effective for detecting this specific attack.

Attack D: Shai-Hulud 2.0

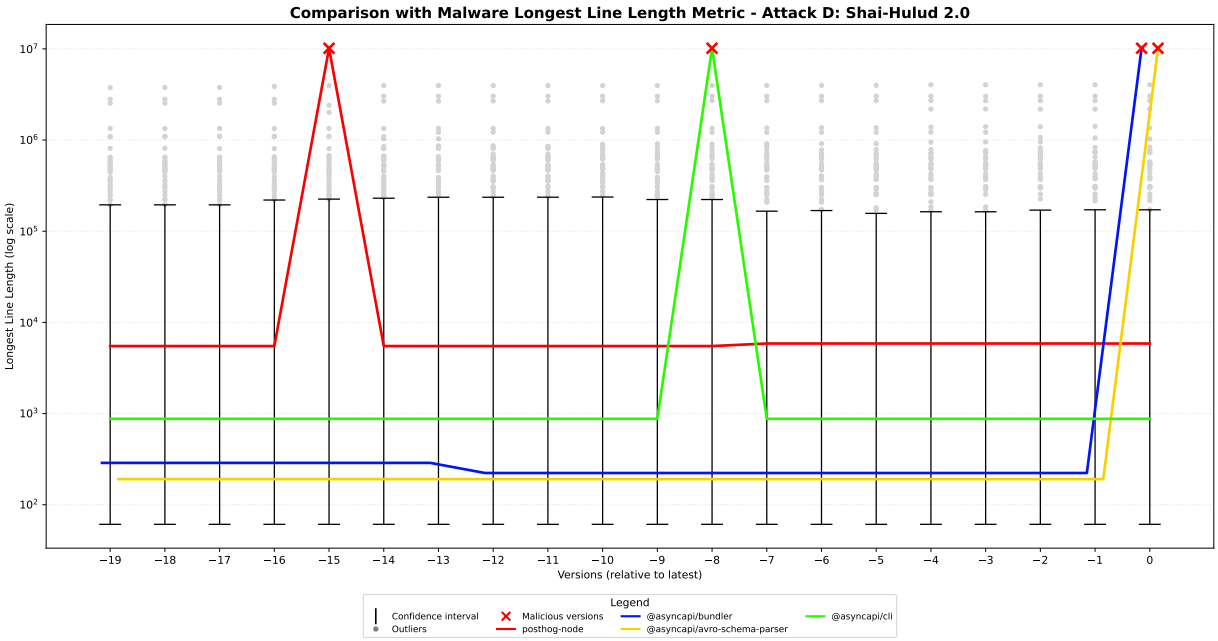

Figure 6.19: Comparison with Malware Dataset Longest Line Length Metric for Attack D

Finally, in the attack D (Section 4.4), the attacker introduces a malicious file named `setup_bun.js` containing a single line of 10,157,373 characters. As shown in Fig. 6.19, the malicious versions not only reach the outlier range but far exceed the maximum outlier values of the Goodware dataset. Therefore, the metric is effective for detecting this attack.

Summary of Comparison with Malware Dataset

Despite this metric being influenced by the presence of build artifacts in the tarballs, it is possible to effectively detect the attack C (`Shai-Hulud` 1.0 Section 6.3.3) and the attack D (`Shai-Hulud` 2.0 Section 6.3.3), because in both cases the malicious versions introduce a malicious line whose length is sufficient to place them among the outlier values.

Instead, the metric is not indicative for detecting the attack A (Malicious Windows DLL Section 6.3.3) and the attack B (Crypto attack Section 6.3.3) because in these cases the malicious versions introduce a malicious line that is not long enough to place them among the outlier values, rendering them indistinguishable from legitimate versions.

6.4 Analysis Metric 4: Unique Hexadecimal Values

6.4.1 Metric Definition

The fourth metric we analyzed is the number of unique hexadecimal values found in the plain text files of the NPM packages.

To compute this metric and identify strings that represent hexadecimal values, the developed tool (Section 5.2.1) uses the following pattern regular expression:

```
_?0x[0-9a-fA-F]+
```

The `?` indicates that the string may optionally begin with an underscore, this is followed by `0x` and a sequence of one or more hexadecimal characters, as specified by `[0-9a-fA-F]+` (i.e., digits 0 to 9 and letters a to f, both lowercase and uppercase).

This flexible pattern allows the tool to find hexadecimal values even when they are part of function calls or other code structures, where strict boundaries, like those used in Section 6.5.1 for Unique Ethereum Addresses metric, would miss them.

The tool identifies these values in each plain text file of a package version, ignores duplicates within that version to ensure the values are unique, and then sums the counts to obtain the total number of unique hexadecimal values for the entire package version.

This metric was taken into consideration because attacks B (Section 4.2) and D (Section 4.4) introduce obfuscated code that makes extensive use of hexadecimal values to hide malicious operations.

6.4.2 Goodware Dataset Scenario

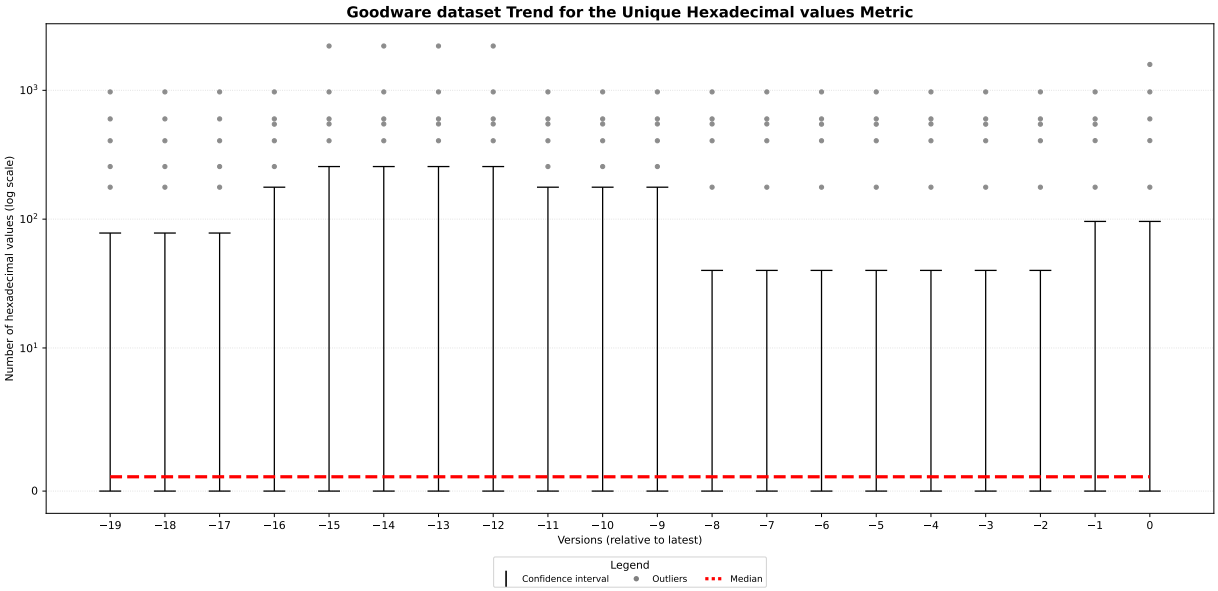

Figure 6.20: Goodware dataset Trend for the Unique Hexadecimal values Metric

For this metric, the Goodware dataset (Section 5.1.1) contains the total count of unique hexadecimal values for each of the 20 versions of every package.

We used four figures, Fig. 6.20 Fig. 6.21 Fig. 6.22 and Fig. 6.23, to visualize the trend of this metric across 20 versions in the Goodware dataset (explained in Section 5.1.1). The characteristics of these figures are described in Section 6.0.1.

These figures report the 109 packages in the Goodware dataset that had at least one positive hexadecimal value in at least one of the 20 versions. The remaining 520 packages in the Goodware dataset did not have any hexadecimal value in any of their 20 versions. Therefore, they were excluded from these figures as they were not significant for this specific metric (they did not have any value to analyze). The median, calculated on the 109 packages, remains constant at one in each version.

We studied the ten packages that had outlier values or values that are the upper limit of a confidence interval for one or more versions.

We highlighted these packages with solid colored lines, categorizing them into the following figures based on their observed trends:

- **Increasing Trend:** Fig. 6.21 shows highlighted packages with an increasing trend

over time.

- **Decreasing Trend:** Fig. 6.22 shows highlighted packages with a decreasing trend.
- **Stable Trend:** Fig. 6.23 shows highlighted packages with a stable trend.

Analysis Outliers Packages: Increasing Trend

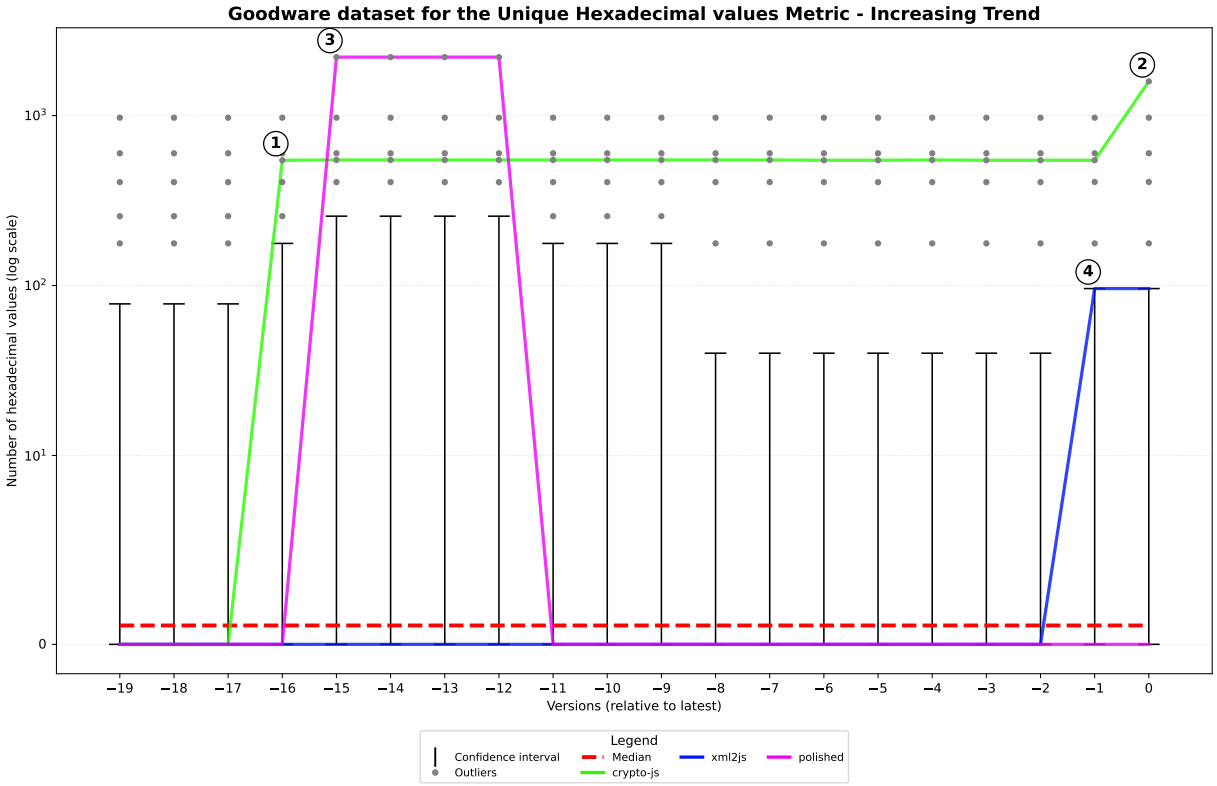

Figure 6.21: Outliers package Increasing Trend for the Unique Hexadecimal Values Metric in Goodware dataset

The packages that experienced an increase significant enough to make their number of unique hexadecimal values an outlier or to be the upper limit of a confidence interval from one version to the next were: `crypto-js`, `polished`, and `xml2js`.

- **crypto-js:** This package is related to cryptography and implements cryptographic primitives, such as encryption algorithms and digital signatures [Mota].

Represented by the solid green line in Fig. 6.21, the package used decimal values to implement its cryptographic primitives in its first three versions (from thirteen years ago, version 3.1.2 to 3.1.2-2). Since it had no hexadecimal values, the metric for these versions was 0 (versions -19 to -17).

In the subsequent release (3.1.2-3), as shown at point 1 in Fig. 6.21, the developers converted these values into hexadecimal format to align with standard cryptographic implementations practices (like Secure Hash Algorithm (SHA) [GD95]). This update increased the number of unique hexadecimal values to 546, rapresenting an outlier also for subsequent versions. The further increase visible at point 2 is due to the introduction of the `Blowfish` encryption algorithm in version 4.2.0, where the count of unique hexadecimal values reached 1588, representing the most extreme outlier for the latest version.

- **Polished:** Represented by the solid pink line in Fig. 6.21, `polished` is used for styling in JavaScript [HS]. As seen at point 3, it had a massive increase in unique hexadecimal values, jumping from 0 to 2208 in version 4.0.0-beta.2, representing the most extreme outlier for four consecutive versions. In these versions, as mentioned in Section 6.2.2, the developers erroneously included a `.yarn` cache directory in the final tarball distributed to the public. The directory contained numerous files, including a generated bundle JavaScript file containing many hexadecimal values.

  In version 4.0.4 the folder was removed from the tarball, as reported in the release notes on GitHub [HS25], and the number of unique hexadecimal values dropped back to 0 (version -11).

- **xml2js:** Rappresented by the solid blue line in Fig. 6.21, `xml2js` had a considerable increase at point 4, moving from 0 to 96 unique hexadecimal values.

  This increase occurred because, starting from version 0.6.1, the developers included a generated JavaScript script in the package. The file was originally written in OCaml, but to ensure its execution in Node.js environments, the code was automatically converted to JavaScript using the `js_of_ocaml` compiler. For efficiency, the compilation process produces a script that makes extensive use of hexadecimal constants. The generated JavaScript file remained in the package tarball for the subsequent version 0.6.2.

Analysis Outliers Packages: Decreasing Trend

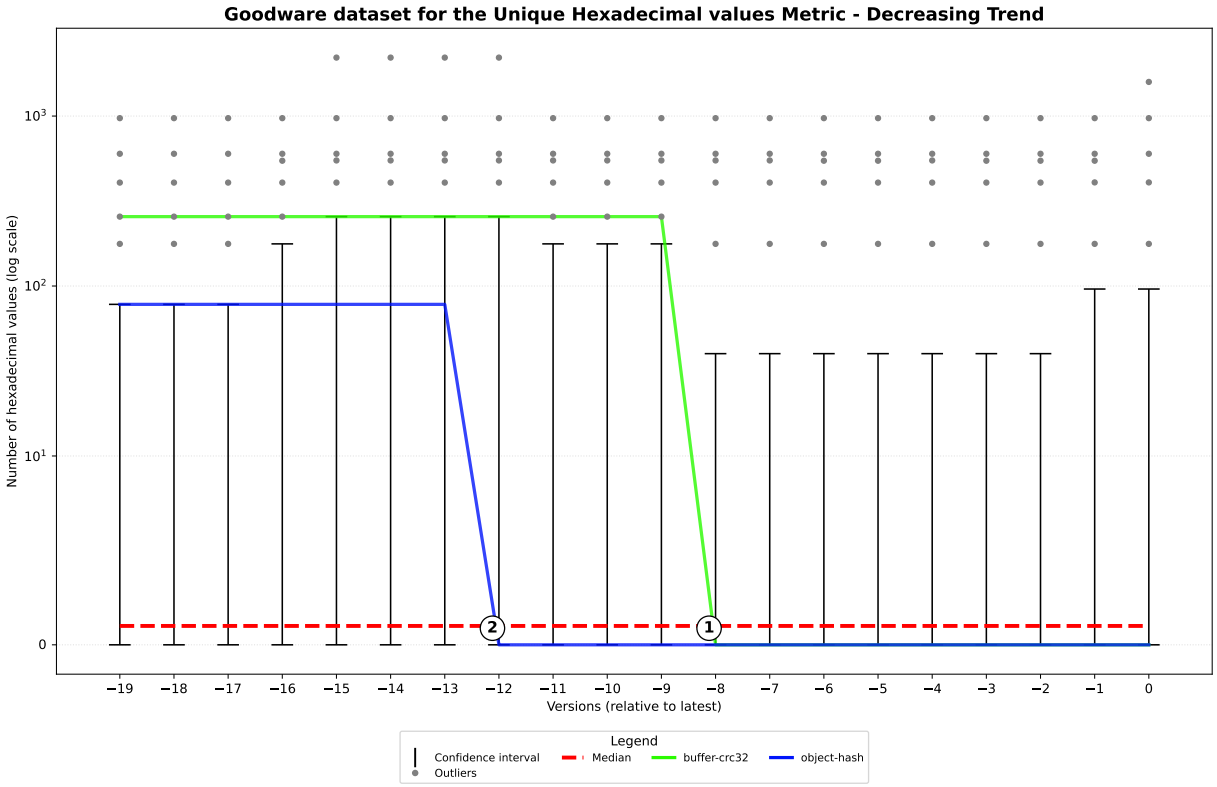

Figure 6.22: Outliers package Decreasing Trend for the Unique Hexadecimal Values Metric in Goodware dataset

The packages that had a considerable decrease from one version to the next are: `buffer-crc32` and `object-hash`.

- **buffer-crc32:** The `buffer-crc32` package implements the Cyclic Redundancy Check (CRC32) algorithm, primarily used to detect accidental errors in data transmission or storage [Bre]. It is represented by the solid green line in Fig. 6.22.

  To make the data integrity check fast and efficient, the algorithm uses a CRC table, a matrix of 256 pre-calculated values that prevents the processor from performing complex mathematical calculations on every single byte. Specifically, until version 0.2.13 (the version before point 1 in Fig. 6.22), the package stored these CRC table values in hexadecimal format.

As shown at point 1 in Fig. 6.22, starting from version 1.0.0-RC1, the developers optimized the package space by including in the tarball the CRC table file generated using automatic build tools, such as minifiers. In this automated process, these build tools choose to convert the hexadecimal values into decimal format because it is more compact. While the mathematical value of the constants remained unchanged, the number of unique hexadecimal values dropped from 256 to 0.

- **object-hash:** The `object-hash` package generates a hash from JavaScript objects and values [Hen]. In Fig. 6.22 (solid blue line), it can be observed that from version -19 to -17 (1.1.0 to 1.1.2), the package represented the upper limit of the confidence intervals for those versions, having 78 unique hexadecimal values. These values were all located in a test file, `object_hash_test.js`, which contained various tests to verify the package's correct operation.

  As shown at point 2 in Fig. 6.22, starting from version 1.1.7, the developers optimized the package space by excluding the test file from the tarball, as it is not essential for its functionality. The removal of this file caused the number of unique hexadecimal values to drop from 78 to 0.

Analysis Outliers Packages: Stable Trend

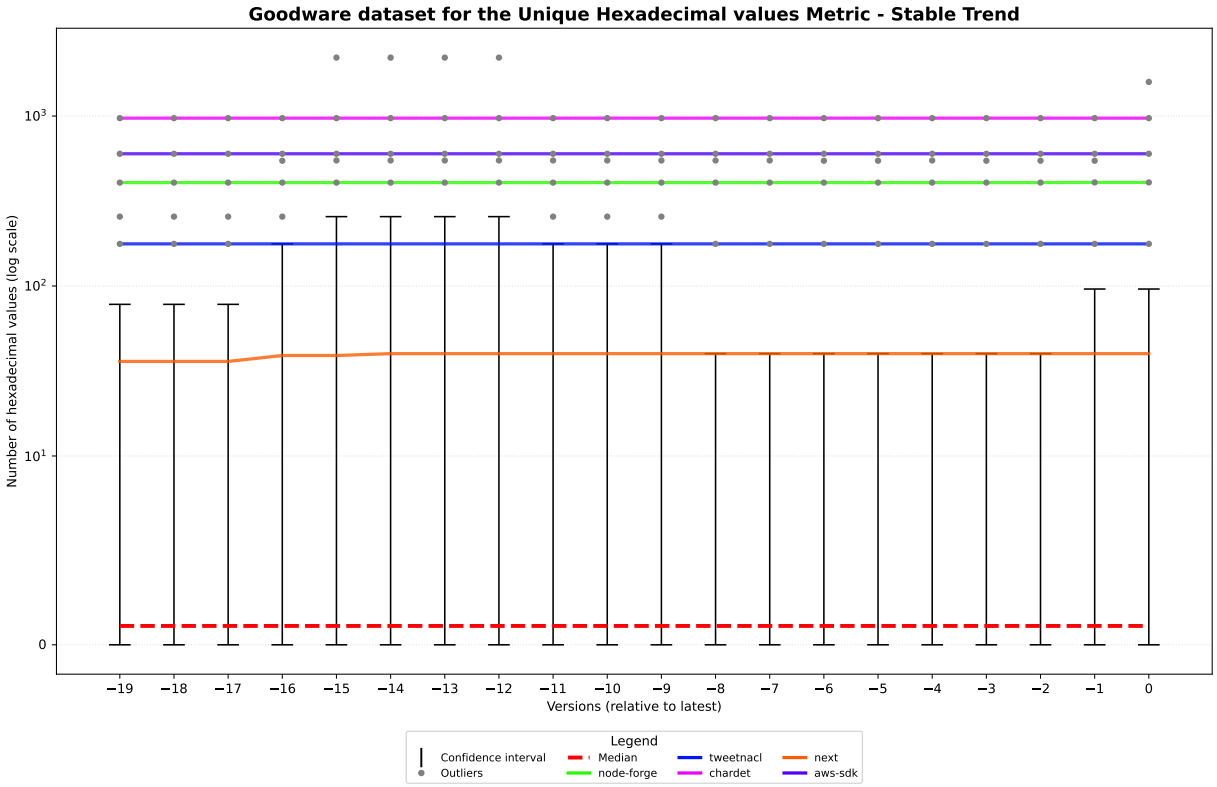

Figure 6.23: Outliers package Stable Trend for the Unique Hexadecimal Values Metric in Goodware dataset

The remaining 5 highlighted packages had outlier values or values representing the upper limit of the confidence intervals that remained constant across all 20 versions, without significant increases or decreases. The packages exhibiting this trend are: `node-forge`, `tweetnacl`, `chardet`, `next`, and `aws-sdk`.

- **node-forge and tweetnacl:** Both packages are related to cryptography [Dig; Che] and implement cryptographic primitives that use hexadecimal constants for their internal functioning, like `crypto-js` seen in Section 6.4.2.

  In Fig. 6.23, `node-forge` (solid green line) has 406 unique hexadecimal values, representing an outlier across all versions; while `tweetnacl` (solid blue line) has 177 unique hexadecimal values, representing the upper limit of the confidence intervals

for versions -16 and from -11 to -9, and representing an outlier for the remaining versions.

- **chardet:** This package detects character encoding, and uses tables and byte maps in the form of hexadecimal constants, primarily located in the file `/lib/encoding/sbcs.js` [Yea].

  Represented in Fig. 6.23 by the solid pink line, the package assumes 972 unique hexadecimal values, representing the most extreme outlier for most versions. It was surpassed only by the `polished` package from version -15 to version -12, and in the latest version (0) by the `crypto-js` package.

- **next and aws-sdk:** Finally, the `next` and `aws-sdk` packages contain numerous JavaScript files in their tarballs that are generated by build tools and containing hexadecimal values. This behavior is the same as seen for the `polished` in Section 6.4.2 and `buffer-crc32` packages in Section 6.4.2.

  In detail: the `next` package, a React full-stack framework [Ver] represented by the solid orange line in Fig. 6.23, has 40 unique hexadecimal values consistently across all versions, representing the upper limit of the confidence intervals for versions from -8 to -2. Conversely, `aws-sdk`, the official package for interacting with AWS cloud services [Ama] represented by the solid purple line, has 600 unique hexadecimal values consistently across all versions, representing an outlier for all versions.

6.4.3 Comparison with Malware Dataset

We analyzed when this metric is effective for detecting these four attacks (described in Chapter 4) and when it is not indicative. To do so, we used a specific figure for each attack, Fig. 6.24, Fig. 6.25, Fig. 6.26 and Fig. 6.27, as explained in Section 6.0.2.

Attack A: Malicious Windows DLL

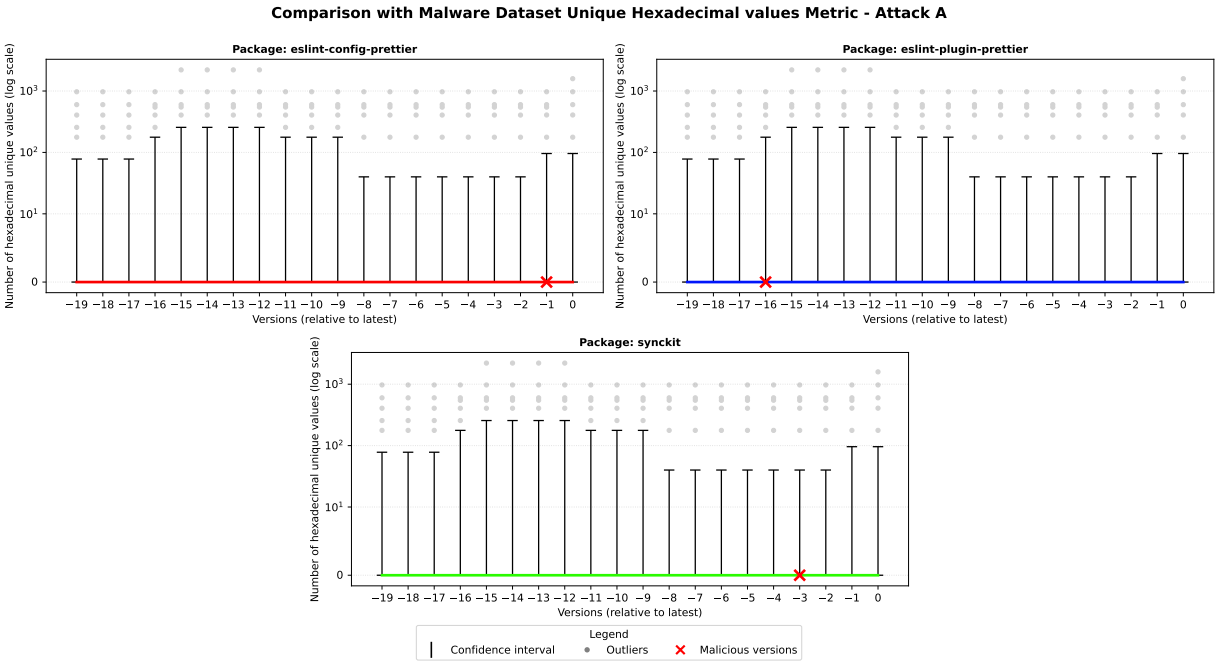

Figure 6.24: Comparison with Malware Dataset Unique Hexadecimal Values Metric for Attack A

In attack A (Section 4.1), the attacker did not introduce any new hexadecimal values. Therefore, the metric remained unchanged compared to previous versions, as shown in Fig. 6.24. Consequently, this metric is not indicative for detecting this specific attack.

Attack B: Crypto attack

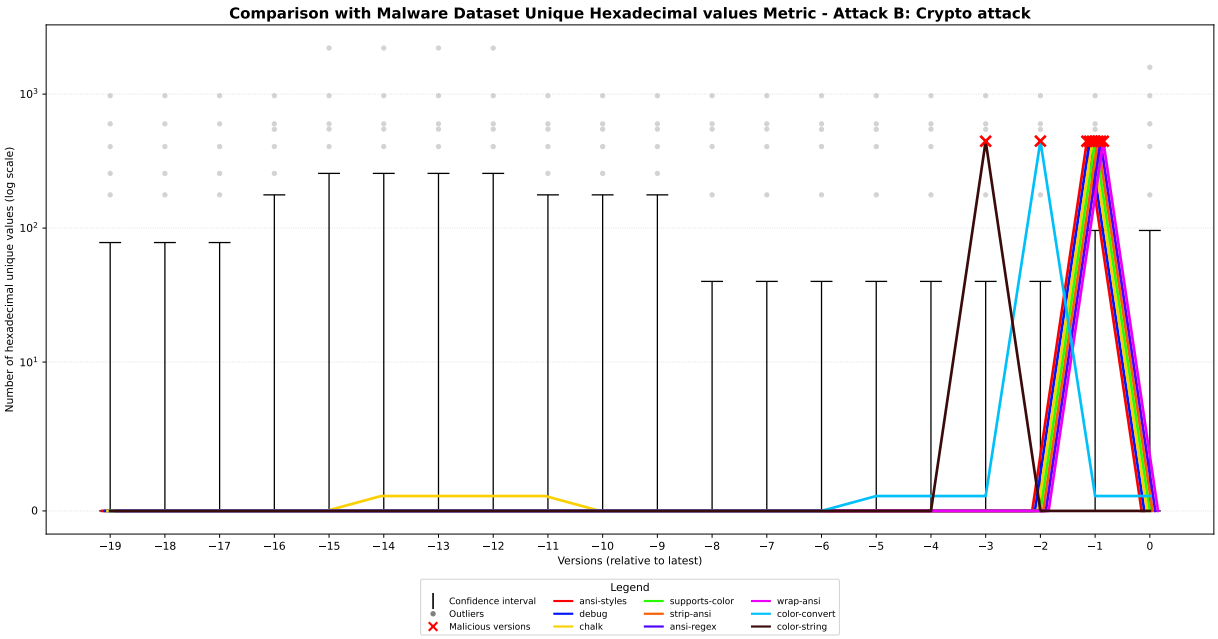

Figure 6.25: Comparison with Malware Dataset Unique Hexadecimal Values Metric for Attack B

In the attack B (Section 4.2), the attacker introduced an obfuscated line of code into an existing file, increasing the number of unique hexadecimal values to 455. As shown in Fig. 6.25, this value positions the malicious versions of the packages among the outlier values. Therefore, the metric is effective for detecting this specific attack.

Attack C: Shai-Hulud 1.0

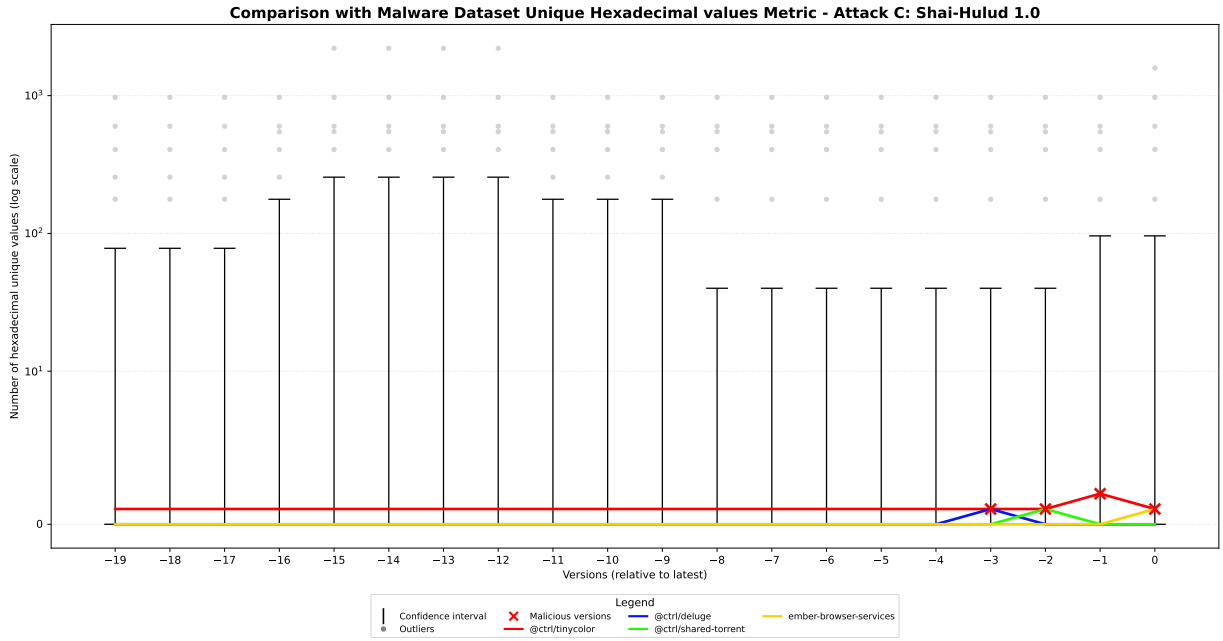

Figure 6.26: Comparison with Malware Dataset Unique Hexadecimal Values Metric for Attack C

In the attack C (Section 4.3), the attacker introduced a malicious clear-text file containing only one unique hexadecimal value. As shown in Fig. 6.26, this marginal increase is insufficient to position the malicious versions of the packages outside the confidence intervals. Consequently, the metric is not indicative for detecting this specific attack, as the compromised versions remain indistinguishable from legitimate ones.

Attack D: Shai-Hulud 2.0

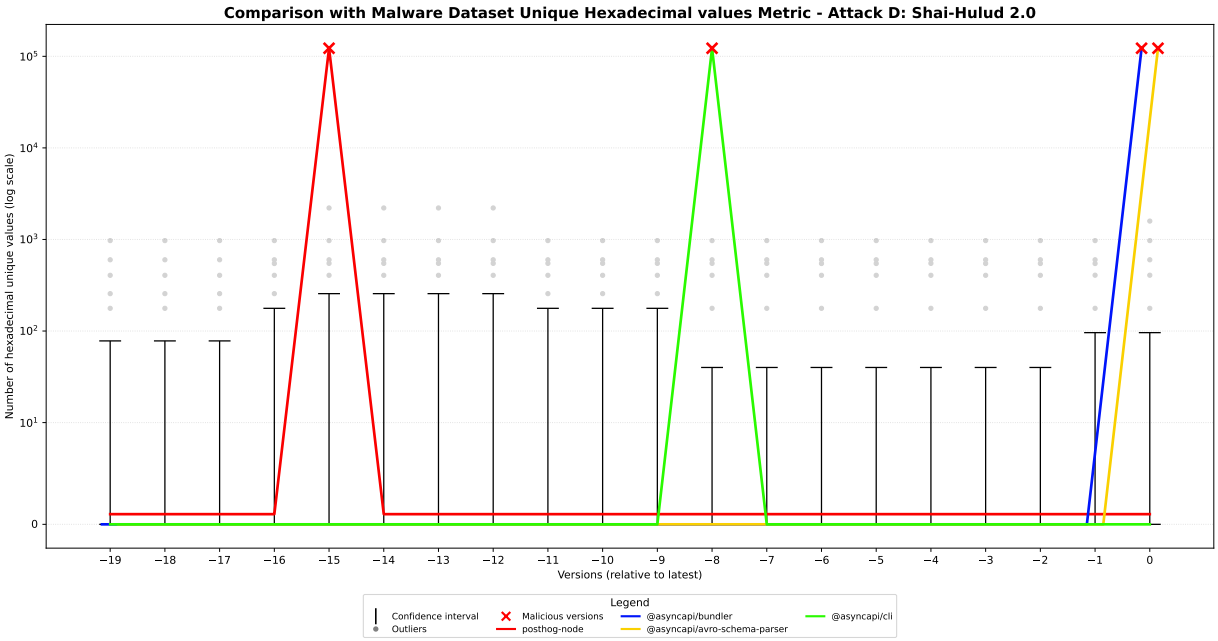

Figure 6.27: Comparison with Malware Dataset Unique Hexadecimal Values Metric for Attack D

Finally, in the attack D (Section 4.4), the attacker introduces an obfuscated malicious file. As shown in Fig. 6.27, the packages with the compromised version have a number of unique hexadecimal values that skyrockets to 122,535, far exceeding the maximum outlier values. Therefore, the metric is effective for detecting this attack.

Summary of Comparison with Malware Dataset

In summary, the metric is efficient in detecting the attack B (Crypto attack Section 6.4.3) and the attack D (`Shai-Hulud` 2.0 Section 6.4.3) because, in both cases, the malicious versions have a number of unique hexadecimal values that qualify as outlier values.

Instead, the metric is not indicative for detecting the attack A (Malicious Windows DLL Section 6.4.3) and the attack C (`Shai-Hulud` 1.0 Section 6.4.3) because, in both cases, the malicious versions have a number of unique hexadecimal values that position them within the confidence intervals, rendering them indistinguishable from legitimate versions.

6.5 Analysis Metric 5: Unique Ethereum Addresses

6.5.1 Metric Definition

The last metric we analyzed is the number of unique Ethereum addresses found in the plain text files of the NPM packages.

To compute this metric and identify strings that represent Ethereum addresses, the developed tool (Section 5.2.1) uses the following precompiled regular expression:

```
(?<![\w\-/_])0x[a-fA-F0-9]{40}(?![\w\-/_])
```

The regex uses a lookbehind `(?<![\w\-/_])` and a lookahead `(?![\w\-/_])` to ensure that the address is neither preceded nor followed by words, hyphens, underscores or slashes. This avoids, for example, capturing a string that is part of a URL or a file path, reducing false positives. Unlike word boundaries `\b`, this solution also correctly handles the typical path separators, like slashes and hyphens.

A valid Ethereum address consists of exactly 40 hexadecimal characters (i.e., digits 0 to 9 and letters a to f, both lowercase and uppercase) preceded by the prefix `0x` [BC14].

The tool identifies all occurrences of this pattern in each plain text file of a package version, ignores duplicates within that version to ensure the values are unique, and then sums the counts to obtain the total number of unique Ethereum addresses for the entire package version.

This metric was taken into consideration because attacks B (Section 4.2) introduce seven smart contract Ethereum addresses.

6.5.2 Goodware Dataset Scenario

For this metric, the Goodware dataset (Section 5.1.1) contains the total count of unique Ethereum addresses for each of the 20 versions of every package.

For this metric we do not present any figure because in the Goodware dataset only two packages have exactly one Ethereum address in at least one version, namely `brace-expansion` and `dotenv-expand`. We analyze both packages to understand the reason for this presence.

- **brace-expansion:** This package provides brace expansion for JavaScript, allowing the use of shell-like syntax to generate sets of strings from a pattern [Gru].

  Starting from version 4.0.0 (the second-to-last available version), the tarball includes the file `tea.yaml`, a configuration file required to register an open-source project on

the Tea protocol. The Tea protocol is an incentivization protocol for open-source projects, which allows registered projects to earn TEA tokens based on their popularity and usage [Inc]. The `tea.yaml` file must contain one or a list of Ethereum addresses to receive TEA tokens and to define who controls the project. The file is also present in the latest version 4.0.1.

- **dotenv-expand:** This package extends the functionality of dotenv by allowing variable expansion in dotenv files [Motb].

  Similarly to `brace-expansion`, starting from version 11.0.7 the tarball includes the file `tea.yaml` containing an Ethereum address. This file remained present for four consecutive versions, until version 12.0.3, the most recent available, in which it was removed.

6.5.3 Comparison with Malware Dataset

Although the metric has a very small reference sample in the Goodware dataset (Section 5.1.1), it proves effective in detecting attack B (Section 4.2). In fact, in the compromised versions of this attack, seven known smart contract Ethereum addresses are introduced into the packages. These addresses are correctly identified by the tool (Section 5.2.1), placing the malicious versions among outlier values.

However, the metric is not indicative for the other three attacks. For attack A (Section 4.1), the malicious DLL introduced in the tarball is not a plain text file and therefore does not contain Ethereum addresses, so the count remains unchanged compared to legitimate versions. For attacks C and D (Section 4.3 and Section 4.4), the malicious code introduced is aimed at stealing credentials and access keys to cloud services, not at manipulating cryptocurrency transactions. Consequently, the malicious files `bundle.js` and `bun_environment.js` do not contain any Ethereum addresses and the count remains unchanged compared to the legitimate versions. Therefore, the malicious versions of these three attacks are indistinguishable from legitimate ones for this metric.

6.6 Wrap up Analysis Metrics

In this section we summarize the results of the analysis conducted on the five metrics of Chapter 6, evaluating their overall effectiveness in detecting the four attacks considered.

Table 6.1: Summary of Metrics Analysis

	Metric 1	Metric 2	Metric 3	Metric 4	Metric 5
Attack A	●	○	○	○	○
Attack B	○	○	○	●	●
Attack C	○	●	●	○	○
Attack D	○	●	●	●	○

Table 6.1 visually summarizes the detection capability of each metric for every attack. A filled green circle (●) indicates that the metric proved effective in detecting the attack, while an empty red circle (○) indicates that the metric was not indicative for that attack.

From the analysis it can be observed that every attack is detectable by at least one of the defined metrics, but no single metric is capable of detecting all four attacks simultaneously. This follows from the fact that each metric focuses on specific characteristics that are not necessarily shared across all the malicious versions analyzed.

We report below the effectiveness of each metric.

- **Metric 1 - File Types** (Section 6.1): This metric proves effective in detecting attacks where the attacker introduces or modify file types rarely found in NPM packages, such as PE files, as in the case of Attack A (Section 4.1).

  It is instead not indicative for attacks that introduce or modify file types commonly found in NPM packages, such as JavaScript files, as in the case of Attacks B (Section 4.2), C (Section 4.3), and D (Section 4.4).

- **Metric 2 - Package Size** (Section 6.2): This metric proves effective for attacks that introduce malicious files of considerable size, as in the case of Attacks C (Section 4.3) and D (Section 4.4).

  It is instead not indicative for attacks that introduce smaller files or that make modifications which do not significantly increase the package size, as in the case of Attacks A (Section 4.1) and B (Section 4.2).

- **Metric 3 - Longest Line Length** (Section 6.3): This metric proves effective for attacks that introduce extremely long lines of code, as in the case of Attacks C (Section 4.3) and D (Section 4.4).

It is instead not indicative for attacks that introduce shorter lines of code or that do not modify any plain text file, as in the case of Attacks A (Section 4.1) and B (Section 4.2).

- **Metric 4 - Unique Hexadecimal Values** (Section 6.4): This metric proves effective for attacks that introduce obfuscated code making extensive use of hexadecimal values to encode variable and function names, such as in the case of Attacks B (Section 4.2) and D (Section 4.4).

  It is instead not indicative for attacks that do not introduce obfuscated code, as in the case of Attacks A (Section 4.1) and C (Section 4.3).

- **Metric 5 - Unique Ethereum Addresses** (Section 6.5): This metric proves effective for attacks whose goal falls within the domain of cryptojacking and wallet theft, as in the case of Attack B (Section 4.2).

  It is instead not indicative for attacks that have different purposes, as in the case of Attacks A (Section 4.1), C (Section 4.3), and D (Section 4.4).

Chapter 7

Conclusion

Bibliography

[Ama] Inc. Amazon Web Services. *Package aws-sdk*. URL: https://www.npmjs.com/package/aws-sdk.

[Bre] Brain J. Brennan. *Package buffer-crc32*. URL: https://www.npmjs.com/package/buffer-crc32.

[Bru] Max Brunsfeld. *Tree-sitter*. URL: https://tree-sitter.github.io/tree-sitter/.

[BC25] Moshe Siman Tov Bustan and Alexander Chailytko. *180+ NPM Packages Hit in Major Supply Chain Attack*. 2025. URL: https://www.ox.security/blog/npm-2-0-hack-40-npm-packages-hit-in-major-supply-chain-attack/.

[BC14] Vitalik Buterin and Ethereum Contributors. *Ethereum Documentation*. 2014. URL: https://ethereum.org/learn/.

[Che] Dmitry Chestnykh. *Package tweetnacl*. URL: https://www.npmjs.com/package/tweetnacl.

[Con] Mozilla Contributors. *Lexical grammar JavaScript*. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Lexical_grammar.

[con] Node.js contributors. *Package node-addon-api*. URL: https://www.npmjs.com/package/node-addon-api.

[Cut25] Gianpietro Cutolo. *Shai-Hulud 2.0: Aggressive, Automated, and Fast Spreading*. 2025. URL: https://www.netskope.com/blog/shai-hulud-2-0-aggressive-automated-one-of-fastest-spreading-npm-supply-chain-attacks-ever-observed.

[Dat23] Datadog. Mar. 2023. URL: https://github.com/datadog/malicious-software-packages-dataset.

[Def] DefinitelyTyped. *Package @types/node*. URL: https://www.npmjs.com/package/@types/node.

[Dig] Inc. Digital Bazaar. *Package node-forge*. URL: https://www.npmjs.com/package/node-forge.

[Dua+20] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. *Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages*. 2020. arXiv: 2002.01139 [cs.CR]. URL: https://arxiv.org/abs/2002.01139.

[Eer] Matthew Eernisse. *Package Jake*. URL: https://www.npmjs.com/package/jake.

[Fou] Python Software Foundation. *Python re Library*. URL: https://docs.python.org/3/library/re.html.

[Ful] Lovell Fuller. *Package sharp*. URL: https://www.npmjs.com/package/sharp.

[GD95] Patrick Gallagher and Acting Director. "Secure hash standard (shs)". In: *Fips pub* 180 (1995), p. 183.

[Gil26] Patrick Gillespie. *Figlet Release Notes*. 2026. URL: https://github.com/patorjk/figlet.js/releases.

[Gil] Patrick Gillespie. *Package figlet*. URL: https://www.npmjs.com/package/figlet.

[Goo] Google. *Magika*. URL: https://github.com/google/magika.

[Gos+20] Pronnoy Goswami, Saksham Gupta, Zhiyuan Li, Na Meng, and Daphne Yao. "Investigating The Reproducibility of NPM Packages". In: *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2020, pp. 677–681. DOI: 10.1109/ICSME46990.2020.00071.

[Gru] Julian Gruber. *Package brace-expansion*. NPM Package Registry. URL: https://www.npmjs.com/package/brace-expansion.

[Guo+23] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. *An Empirical Study of Malicious Code In PyPI Ecosystem*. 2023. arXiv: 2309.11021 [cs.SE]. URL: https://arxiv.org/abs/2309.11021.

[HH25] Asaf Henig and Cameron Hyde. *Breakdown: Widespread npm Supply Chain Attack Puts Billions of Weekly Downloads at Risk*. 2025. URL: https://www.paloaltonetworks.com/blog/cloud-security/npm-supply-chain-attack/.

[Hen] Basim Hennawi. *Package object-hash*. URL: https://www.npmjs.com/package/object-hash.

[HS25] Brian Hough and Max Stoiber. *Polished Release Notes*. 2025. URL: https://github.com/styled-components/polished/releases?page=2.

[HS] Brian Hough and Max Stoiber. *Package polished*. URL: https://www.npmjs.com/package/polished.

[Hua+24a] Cheng Huang, Nannan Wang, Ziyan Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. *DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping*. 2024. arXiv: 2403.08334 [cs.CR]. URL: https://arxiv.org/abs/2403.08334.

[Hua+24b] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. "SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching". In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ASE '24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 1146–1158. ISBN: 9798400712487. DOI: 10.1145/3691620.3695492. URL: https://doi.org/10.1145/3691620.3695492.

[Inc] Tea Inc. *Tea Protocol Documentation*. URL: https://docs.tea.xyz/tea.

[Lab25] Endor Labs. *CVE-2025-54313: eslint-config-prettier Compromise — High Severity but Windows-Only*. 2025. URL: https://www.endorlabs.com/learn/cve-2025-54313-eslint-config-prettier-compromise----high-severity-but-windows-only.

[Mat25] Nicolas Matthee. *The npm Supply Chain Attack of September 2025*. 2025. URL: https://www.prventi.com/blog/the-npm-supply-chain-attack-of-september-2025.

[Mic] Microsoft. *Package typescript*. URL: https://www.npmjs.com/package/typescript.

[Moo+21] Marvin Moog, Markus Demmel, Michael Backes, and Aurore Fass. "Statically Detecting JavaScript Obfuscation and Minification Techniques in the Wild". In: *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2021, pp. 569–580. DOI: 10.1109/DSN48987.2021.00065.

[Mota] Jeff Mott. *Package crypto-js*. URL: https://www.npmjs.com/package/crypto-js.

[Motb] Scott Motte. *Package dotenv-expand*. NPM Package Registry. URL: https://www.npmjs.com/package/dotenv-expand.

[npm26] Inc. npm. *npm Documentation*. 2026. URL: https://docs.npmjs.com/.

[Ohm+20] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. *Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks*. 2020. arXiv: 2005.09535 [cs.CR]. URL: https://arxiv.org/abs/2005.09535.

[OS23] Marc Ohm and Charlene Stuke. "Sok: Practical detection of software supply chain attacks". In: *Proceedings of the 18th International Conference on Availability, Reliability and Security*. 2023, pp. 1–11.

[Pat+25] Jinish Patel, Joseph Reiner, Brenden Stilwell, Abdullah Wahbeh, and Raed Seetan. "Leveraging K-Means Clustering and Z-Score for Anomaly Detection in Bitcoin Transactions". In: *Informatics* 12 (Apr. 2025), p. 43. DOI: 10.3390/informatics12020043.

[Pin+23] Donald Pinckney, Federico Cassano, Arjun Guha, and Jonathan Bell. *A Large Scale Analysis of Semantic Versioning in NPM*. 2023. arXiv: 2304.00394 [cs.SE]. URL: https://arxiv.org/abs/2304.00394.

[Pre] Tom Preston-Werner. *Semantic Versioning 2.0.0*. URL: https://semver.org/.

[Sec] Truffle Security. *TruffleHog*. URL: https://trufflesecurity.com/trufflehog.

[Sha] Remy Sharp. *Package nodemon*. URL: https://www.npmjs.com/package/nodemon.

[Sum26] Jarred Sumner. *Bun*. 2026. URL: https://bun.com/.

[Tea] Angular Team. *Package @angular/compiler*. URL: https://www.npmjs.com/package/@angular/compiler.

[tea26] Date-fns team. *Date-fns Release Notes*. 2026. URL: https://github.com/date-fns/date-fns/releases.

[tea] Date-fns team. *Package date-fns*. URL: https://www.npmjs.com/package/date-fns.

[Tea25a] Editorial Team. *npm registry attacked by secret-stealing worm*. 2025. URL: https://www.kaspersky.com/blog/tinycolor-shai-hulud-supply-chain-attack/54315/.

[Tea25b] HelixGuard Team. *Shai-Hulud Returns: Over 1K NPM Packages and 27K+ Github Repos infected via Fake Bun Runtime Within Hours*. 2025. URL: https://helixguard.ai/blog/malicious-sha1hulud-2025-11-24/.

[Tob] Christer Tobjörk. *Package node-releases*. URL: https://www.npmjs.com/package/node-releases.

[Tri26] F.-R. Tristan. *List of top 10000 NPM packages*. 2026. URL: https://tristan-f-r.github.io/npm-rank/PACKAGES.html.

[Typ] Typicode. *Package husky*. URL: https://www.npmjs.com/package/husky.

[VB] Rod Vagg and Benjamin Byholm. *Package nan*. URL: https://www.npmjs.com/package/nan.

[Ver] Vercel. *Package next*. URL: https://www.npmjs.com/package/next.

[Wan+25] Jian Wang, Zhen Li, Jixiang Qu, Deqing Zou, Shouhuai Xu, Ziteng Xu, Zhenwei Wang, and Hai Jin. "MalPacDetector: An LLM-based Malicious NPM Package Detector". In: *IEEE Transactions on Information Forensics and Security* PP (Jan. 2025), pp. 1–1. DOI: 10.1109/TIFS.2025.3580336.

[Wat] Shinnosuke Watanabe. *Package spdx-license-ids*. URL: https://www.npmjs.com/package/spdx-license-ids.

[Wei+25] Felix Weissberg, Lukas Pirch, Erik Imgrund, Jonas Möller, Thorsten Eisenhofer, and Konrad Rieck. "LLM-based Vulnerability Discovery through the Lens of Code Metrics". In: *arXiv preprint arXiv:2509.19117* (2025).

[Yea] Graeme Yeates. *Package chardet*. URL: https://www.npmjs.com/package/chardet.

[Yee] Jecelyn Yeen. *What are source maps?* URL: https://web.dev/articles/source-maps.

[Zah+25] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. *Leveraging Large Language Models to Detect npm Malicious Packages*. 2025. arXiv: 2403.12196 [cs.CR]. URL: https://arxiv.org/abs/2403.12196.

[Zha+25] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. "Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence". In: *ACM Transactions on Software Engineering and Methodology* 34.4 (Apr. 2025), pp. 1–28. ISSN: 1557-7392. DOI: 10.1145/3705304. URL: http://dx.doi.org/10.1145/3705304.

[Zor25] Marco Zoratti. "Valutazione di Metriche per il Rilevamento di Attacchi al Codice Sorgente". In: (2025). URL: https://unire.unige.it/handle/123456789/12801.